

西安交通大学

数字电子技术综合实验报告

题 目 基于 FPGA 的 DDS 波形发生器设计

物理 学院 物理 系

学生姓名 索定扬 班级 强基物理 2103 学号 2216110081

指导教师 孙敏

2024 年 06 月

实验题目：基于 FPGA 的 DDS 波形发生器设计

学生姓名：索定扬

指导教师：孙敏

摘 要

本文详细介绍了一种基于 FPGA 的 EGO1 开发板的 DDS 波形发生器，并通过按键进行波形转换、波形参数调节。

设计内容主要包括三大部分：一是通过开发板上的 5 个按键和有限状态机对于输出波形以及频率、幅值、直流偏移等参数的使能信号进行控制；二是利用 XADC 模数转换将开发板上轮盘电位信号转换为数字信号，并作为参数改变的输入信号；三是利用 DDS 自动波形发生原理，产生方波、正弦波以及三角波。并通过板载数模转换芯片 DAC0832 将数字信号转换为模拟信号输出。最后，通过数码管对于波形、参数、单位等信息进行显示。

关 键 词：FPGA；DDS 波形发生器；有限状态机；DAC0832；XADC 模数转换

目 录

1 前 言	1
1.1 设计背景	1
1.2 主要实现的功能	1
1.3 设计思路	2
1.4 设计难点与亮点	2
1.4.1 设计难点	2
1.4.2 设计亮点	2
2 系统设计	3
2.1 总体结构及功能设计	3
2.1.1 实验平台	3
2.1.1 实验设计流程	3
2.1.2 输入输出设计	4
2.2 系统结构	5
2.2.1 子模块划分	5
2.2.2 顶层 RTL 逻辑图	5
2.3 子模块设计思路	6
2.3.1 按键与状态机模块设计思路	6
2.3.2 顶层波形模块	7
2.3.3 模数转换 XADC 模块与采样脉冲模块	8
2.3.4 波形产生模块	9
2.4 硬件占用	12
3 系统调试	13
3.1 遇到的问题	13
3.1.1 系统出现问题排查错误	13
3.1.2 修改不同参数时，不同参数的相互影响问题	13
3.1.3 基于 DDS 波形发生原理无法改变方波占空比	14
3.1.4 芯片 DAC0832 只输出高电平，无法进行电压转换	14
3.1.5 数码管不亮无输出	14
3.2 解决方案	15
3.2.1 设置多层模块分别仿真、调试	15
3.2.2 选择 5 个按键代替原来的 2 个按键	15

3.2.3	使用采样脉冲, 改变方波占空比	15
3.2.4	必须要对于控制信号在约束文件中定义	16
3.2.5	数码管的输出信号无法复用	16
3.3	创新设计	16
3.3.1	采用 XADC 进行参数的输入	16
3.3.2	数码管显示时设计了修改参数时数码管闪动	16
3.3.3	利用 LED 灯与数码管的结合显示波形信息	17
4	系统使用说明	18
4.1	数码管	18
4.1.1	数码管显示	18
4.1.2	LED 灯显示	18
4.2	按键控制	18
4.2.1	按键(5 个)控制	18
4.2.2	按键与电位器配合	19
5	参考文献	20
6	总结	21
6.1	课程收获	21
6.2	心得体会	21
6.3	实验总结	21
7	附 录	23
7.1	源程序	23
7.1.1	顶层模块(top_dds.v)	23
7.1.2	按键与状态机模块(top_key_and_FSM.v)	25
7.1.3	按键 5 个的综合模块(get_5key.v)	26
7.1.4	按键消抖模块(key_debounce.v)	27
7.1.5	有限状态机模块(FSM_5key.v)	29
7.1.6	顶层波形模块(top_wave.v)	33
7.1.7	模数转换(XADC_to_data.v)	35
7.1.8	模数转换 XADC 的 IP 核模块(XADC_channel_1.v)	36
7.1.9	数据转换输出模块(data_output.v)	37
7.1.10	二进制转换为 BCD 码输出在数码管上(bin2BCD.v)	38
7.1.11	数码管显示模块(show_high_low.v)	40
7.1.12	四位数码管显示模块(show_4_digit.v)	41
7.1.13	将 BCD 码转为有波形, 参数, 单位等信息的码(convert.v)	43
7.1.14	采样脉冲模块(pulse.v)	44
7.1.15	数模转换 DAC 波形模块(wave.v)	46

7.1.16 方波产生模块(square_wave.v)	47
7.1.17 正弦波产生模块(sine_wave.v)	49
7.1.18 三角波产生模块(triangle.v)	51
7.1.19 波形选择模块(select.v)	52
7.2 约束文件(XADC_voltage.xdc)	53

1 前言

1.1 设计背景

在现代电子技术的快速发展和应用中，数字信号处理（DSP）技术在各种领域中发挥着重要作用。其中，直接数字频率合成（DDS）技术因其高精度、灵活性和可编程性而备受关注。DDS 波形发生器作为 DDS 技术的一个典型应用，被广泛应用于通信系统、医疗设备、声音处理、测试测量等领域。

本项目旨在设计并实现一个基于现场可编程门阵列（FPGA）的 DDS 波形发生器。通过 FPGA 的灵活性和强大的计算能力，我们能够实现高精度、高速度的波形发生器，并且可以根据具体需求进行灵活的定制和扩展。该波形发生器将具有以下主要特点：

1. 高精度波形生成：利用 FPGA 的高速时钟和精确的计算能力，实现对各种类型的波形（如正弦波、方波、锯齿波等）的高精度生成，满足不同应用场景的需求。

2. 灵活的频率和相位调节：用户可以通过简单的按键与轮盘，实时调节波形的频率和相位，实现波形的动态调节和变化，适应不同的工作环境和要求。

3. 多种交互接口：波形发生器将提供多种人机交互接口，方便对于波形以及各种参数的调节，包括按键、轮盘以及数码管和 LED 灯。

4. 易于集成和扩展：由于采用了 FPGA 作为核心处理器，波形发生器具有较高的灵活性和可编程性，可以轻松集成到各种系统中，并且可以根据需要进行功能扩展和定制开发。

通过本项目的实施，我们将深入理解 DDS 技术的原理和实现方法，掌握 FPGA 在数字信号处理中的应用。

1.2 主要实现的功能

本实验要求设计一个基于 FPGA 的 DDS 波形发生器，用于产生各种模拟信号波形，具有以下功能：

1. 可产生方波，正弦波和三角波，通过按键可以选择要产生的波形。
2. 通过数码管及 LED 灯显示相关信息，如波形类型、频率、幅值、直流偏移、占空比以及他们的单位等。
3. 波形频率可设置为 $100\text{Hz} \sim 10\text{kHz}$ 。
4. 波形幅度可调节为 $0 \sim 3.7\text{V}$ 。
5. 波形可以调节直流偏移，直流偏移最小为 0，最大可以使波形最大值达到 3.7V 。
6. 对于方波而言，可以对占空比调节， $0 \sim 100\%$ 。
7. 利用 XADC 对于参数实现相对连续、快速的调节。

1.3 设计思路

有限状态机以及 DDS 波形发生原理使系统实现的中心思想。本程序主要由四大模块组成：按键和有限状态机控制模块、波形生成模块、XADC 参数设置模块和参数显示模块四个部分。这四个模块体现了程序的四个部分主要功能。

对于基础控制功能，按键和有限状态机控制模块，通过有限状态机的方式实现按键控制波形生成的状态转换。不同的按键操作对应不同的状态，如选择波形类型、启动/停止波形生成等。有限状态机根据按键输入的状态转换规则，控制波形生成模块的工作状态，实现波形生成的启动、停止、切换等功能。

波形生成模块负责根据参数生成相应的波形信号。根据当前设置的波形类型、频率、幅值等参数，生成对应的波形数据。这部分的设计可以采用直接数字频率合成（DDS）等算法，通过数学运算产生相应的波形数据。

对于创意功能，XADC 参数设置模块用于读取外部模数转换器（XADC）的输入，并将输入的模拟信号转换为数字信号。用户可以通过外部接口（如旋钮、电位器等）调节波形生成的参数，如频率、幅值等，XADC 模块负责将这些调节的参数转换为数字信号，输入到 FPGA 中进行处理。

对于进阶补充功能，参数显示模块负责将波形生成的参数显示在数码管、LED 等显示设备上，供用户实时监测和调节。根据波形生成模块产生的波形参数数据，参数显示模块将这些数据转换为合适的格式，并输出到显示设备上显示，方便用户对波形参数进行观察和调节。

通过以上四个模块的设计和协作，可以实现一个功能完善、灵活可控的基于 FPGA 的 DDS 波形发生器，满足不同用户对波形生成的需求，并提供良好的用户交互体验。

1.4 设计难点与亮点

1.4.1 设计难点

1. 方波占空比的调节方法；
2. 按键个数以及状态翻转个数的选择问题；
3. 基于 DDS 波形发生的频率改变；
4. 修改参数时不同参数的相互影响；
5. 遇到其他的一些问题时的调试并排查错误问题。

1.4.2 设计亮点

1. 采用 XADC 进行参数的输入；
2. 数码管显示时设计了修改参数时数码管闪动；
3. 利用 LED 灯与数码管的结合显示波形信息。

2 系统设计

2.1 总体结构及功能设计

2.1.1 实验平台

FPGA (Field Programmable Gate Array), 即“现场可编程逻辑阵列”, 就是可反复变成的逻辑器件(可编程数字电路)。是在 PAL、GAL 等可编程器件的基础上进一步发展的产物。它是作为专用集成电路(ASIC)领域中的一种半定制电路而出现的, 既解决了定制电路的不足, 又克服了原有可编程器件门电路数有限的缺点。

FPGA 器件属于专用集成电路中的一种半定制电路, 是可编程的逻辑阵列, 能够有效的解决原有的器件门电路数较少的问题。FPGA 的基本结构包括可编程输入输出单元, 可配置逻辑块, 数字时钟管理模块, 嵌入式块 RAM, 布线资源, 内嵌专用硬核, 底层内嵌功能单元。

FPGA 基本的组成有三大部分:

1. 可配置逻辑块 (Configurable Logic Blocks, CLB);
2. 输入输出模块 (Input Output Blocks, IOB);
3. 内部互联资源 (Interconnector)。

而本实验采用的是由依元素科技基于 Xilinx Artix-7 FPGA 研发的便携式数模混合基础教学实验平台。EGO1 配备的 FPGA 具有大容量高性能的特点, 能实现较复杂的数字逻辑。该平台拥有丰富的外设以及灵活的通用拓展接口。

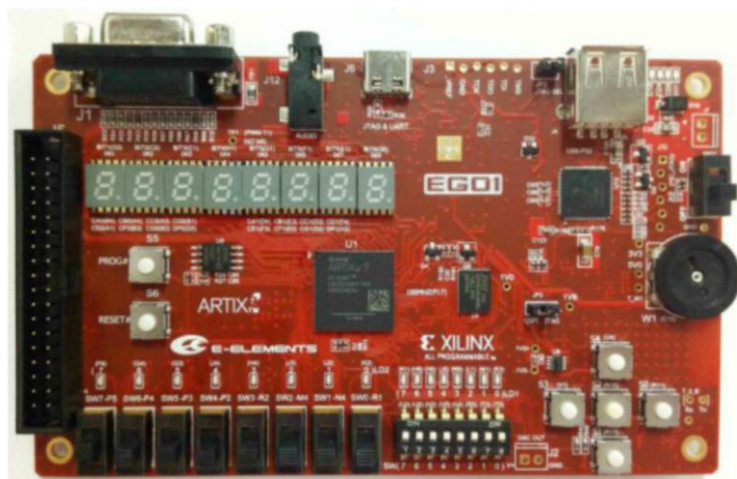


图 2-1 EGO1 开发板

2.1.1 实验设计流程

首先, 设计输入阶段创建 FPGA 工程, 添加源文件等; 然后是设计综合阶段, FPGA 开发工具的综合引擎将编译整个设计, 并将 HDL 源文件转译为特定结构的设计网表;

接下来是约束输入，通过指定番茄钟时序以及布线布局；这些过程中穿插设计仿真，对工程进行功能或时序验证。再之后是设计实现，将逻辑设计进一步转译为目标器件中的特定物理文件格式，即生成 bitstream 文件；然后对结果进行分析，并对资源占用率，结果功耗等进行分析；接着提出设计优化，进行修改，不断优化结果，重新设计综合，约束输入，再一次实现设计，最后板级调试。

2.1.2 输入输出设计

本实验设计了 2 个输入模块，3 个输出模块。

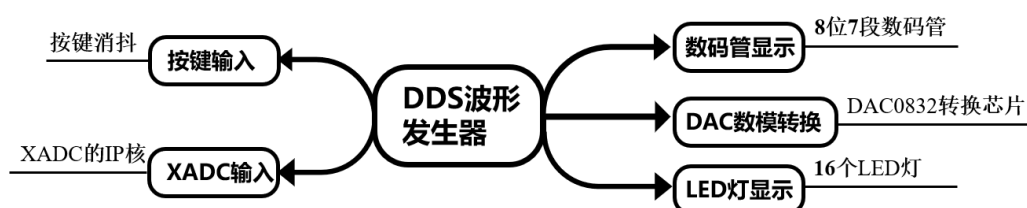


图 2-2 输入输出设计

1. 输入模块：

按键模块：包括五个按键，用于切换波形状态和控制参数改变使能信号。每个按键对应不同的功能，例如切换波形类型、启动/停止波形生成、增加/减少频率等。

模拟电压信号输入模块：用于接收外部模拟电压信号，例如旋钮或电位器产生的模拟电压信号，用于调节波形参数，如频率、相位或幅度。

2. 输出模块：

数码管模块：用于显示当前的波形状态和参数信息，例如显示当前选择的波形类型、频率、相位等。

DAC 模块：数字模拟转换器（DAC）模块，用于将数字信号转换为模拟信号。通过 DAC 模块，生成的模拟信号可以控制外部设备，如示波器、功率放大器等。

LED 指示灯：用于指示当前波形生成器的状态，例如显示是否处于运行状态或停止状态。

2.2 系统结构

2.2.1 子模块划分

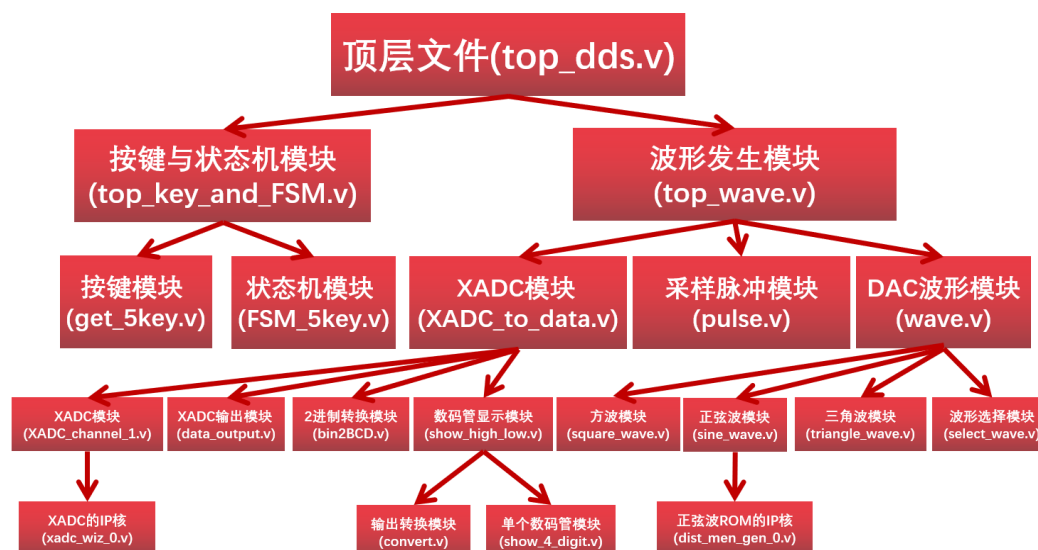


图 2-3 子模块划分示意图

2.2.2 顶层 RTL 逻辑图

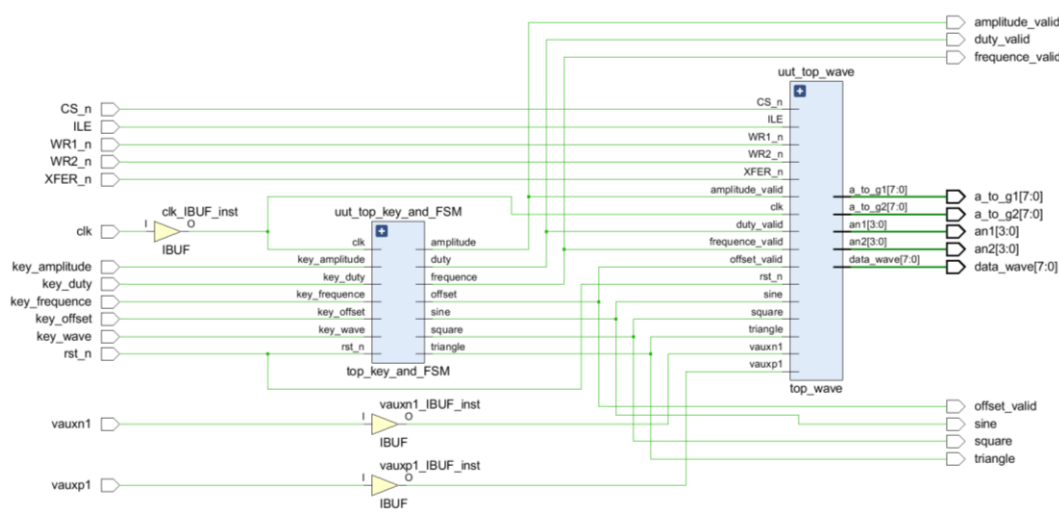


图 2-4 RTL 分析顶层逻辑图

最顶层文件(top.dds.v)，并实例化两个子模块：

按键与状态机模块(top_key_and_FSM.v)，完成对于 5 个按键的消抖与状态的切换控制。

还有波形发生模块(top_wave.v)，根据状态机的输入信号产生波形，改变波形参数，并且完成波形的模数转换与输出。波形发生模块负责根据状态机模块的控制信号生成相应的波形，并将数字波形信号转换为模拟信号输出。

2.3 子模块设计思路

2.3.1 按键与状态机模块设计思路

按键与状态机模块负责处理用户输入的按键信号，并控制系统的状态转换，可分为按键模块(get_5key.v)，有限状态机模块(FSM_5key.v)。如下图所示：

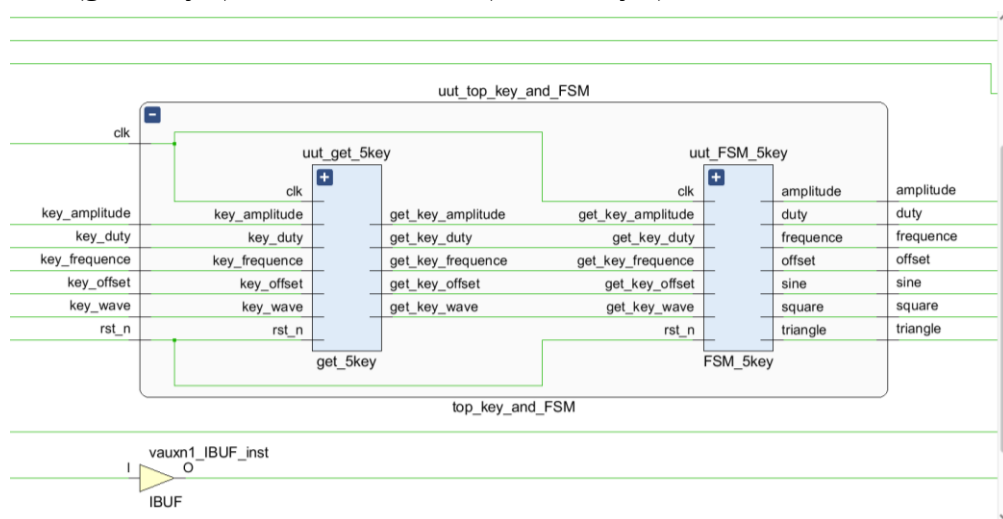


图 2-5 按键与状态机模块 RTL 逻辑图

具体功能包括：

按键模块：分别接受 5 个按键的输入，分别是 key_wave 改变波形按键、key_frequency 改变波形频率按键、key_amplitude 改变波形幅值按键、key_offset 改变波形直流偏移信号以及 key_duty 改变方波占空比按键，具体可见下图：

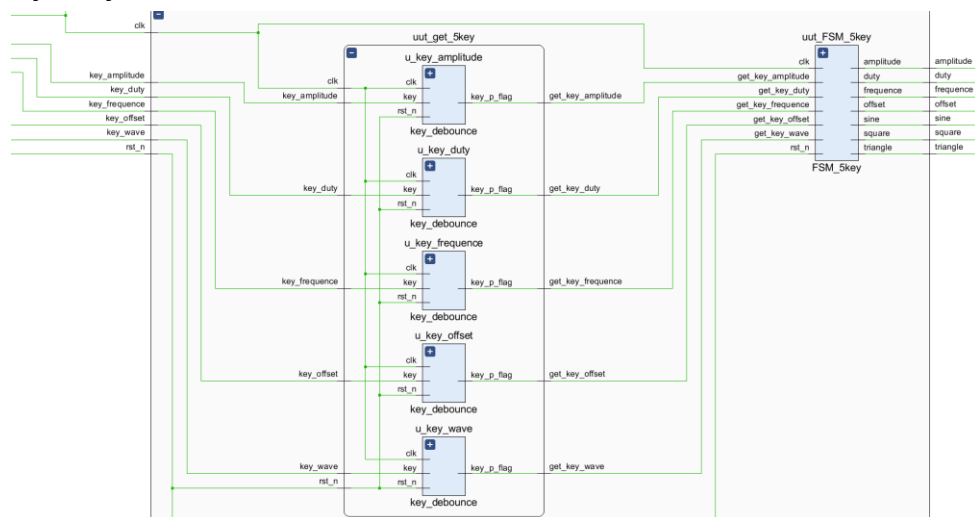


图 2-6 按键模块中实例化的 5 个按键消抖模块

按键消抖：通过 5 个按键消抖模块(key_debounce.v)的实例化，按键信号直接输入往往会产生抖动现象，该模块通过硬件消抖电路对按键信号进行消抖处理，确保按键输入的稳定性与可靠性。

状态机控制：根据按键输入，状态机模块实现波形生成的状态转换和参数设置。不同的按键操作对应不同的状态，如选择波形类型、调整频率、幅值、直流偏移和占

空比等。状态机根据按键输入的状态转换规则，生成相应的控制信号，指导波形发生模块的工作。

不同状态直接通过按键的转换关系如图所示：

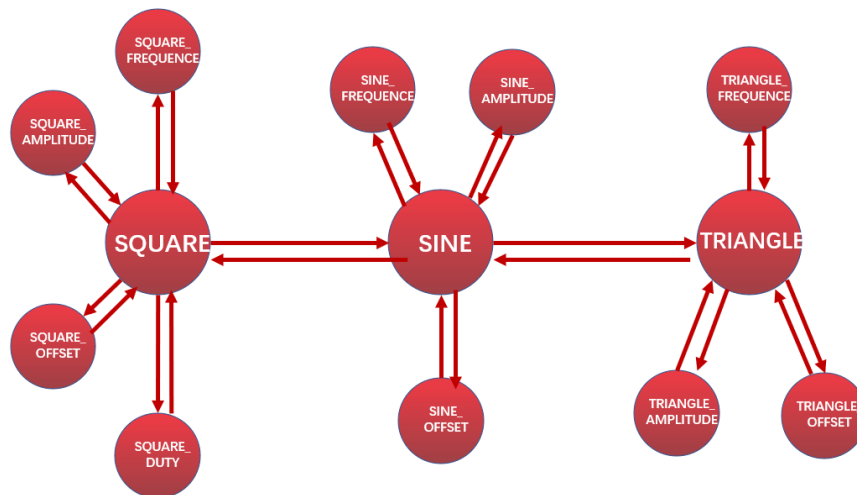


图 2-7 不同状态之间的逻辑转换图

波形发生器共设置了 13 种状态，分别是

- (1) 方波输出状态(SQUARE)
- (2) 方波频率可改变状态(SUQARE_FREQUENCY)
- (3) 方波幅值可改变状态(SQUARE_AMPLITUDE)
- (4) 方波直流偏移可改变状态(SQUARE_OFFSET)
- (5) 方波占空比可改变状态(SQUARE_DUTY)
- (6) 正弦波输出状态(SINE)
- (7) 正弦波频率可改变状态(SINE_FREQUENCY)
- (8) 正弦波幅值可改变状态(SINE_AMPLITUDE)
- (9) 正弦波直流偏移可改变状态(SINE_OFFSET)
- (10) 三角波输出状态(TRIANGLE)
- (11) 三角波频率可改变状态(TRIANGLE_FREQUENCY)
- (12) 三角波幅值可改变状态(TRIANGLE_AMPLITUDE)
- (13) 三角波直流偏移可改变状态(TRIANGLE_OFFSET)

2.3.2 顶层波形模块

顶层波形模块，实例化三个子模块：

XADC 模块(XADC_to_data.v),实现对于电位器的电位模拟信号与数字信号之间的转换，并将数字信号作为波形的参数传给后面的波形模块(wave.v)。

采样脉冲模块(pulse.v)，根据 XADC 转换后的参数，确定需要的波形频率，计算出一个周期的时间，并将一个周期分为 255 份，即：在一个波形周期内发出 255 个采样脉冲，在每一个采样脉冲改变后，保持一定的时间，完成对于数字信号到模拟信号的

转换。

波形转换输出模块(wave.v)，根据传入的 4 个参数有效信号：频率改变有效信号(frequency_valid)，幅值改变有效信号(amplitude_valid)，直流偏移改变有效信号(offset_valid)，占空比改变有效信号(duty_valid)，以及 XADC 输出的参数，对于波形的频率，幅值，直流偏移，占空比进行改变。并利用 DDS 波形生成原理，输出波形。具体的逻辑如下图所示：

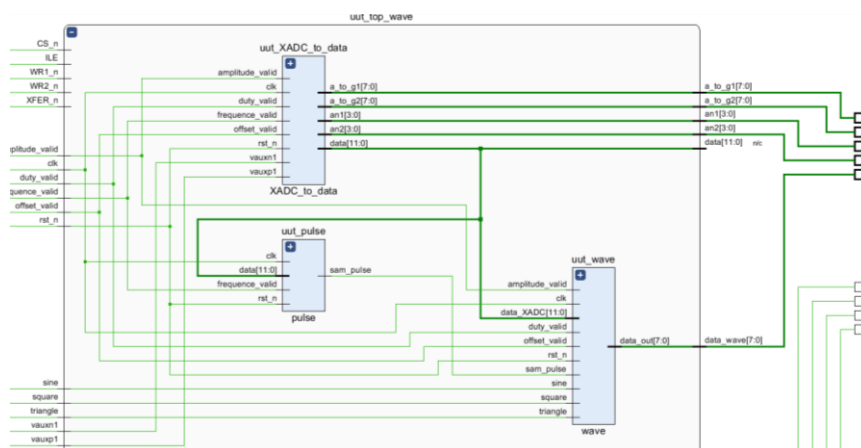


图 2-8 波形发生模块原理图

2.3.3 模数转换 XADC 模块与采样脉冲模块

Xilinx 的 7 系列 FPGA 增加了 XADC 硬核模块，为各种不同的应用提供了通用、高精度的模拟接口。提高系统集成度。XADC 包括了双 12 位，1MSPS (Mega sample per second) 采样率的 ADC，以及片上传感器。在默认模式下，可以采集芯片温度和内部各个供电电压，用于监控 FPGA 内部状况。同时，ADC 具有 17 个外部模拟输入通道（17 对引脚），一对专用模拟差分输入对 VP/VN，16 对可复用的辅助模拟输入引脚 ADXP/ADXN，不使用时可作为通用 I/O 使用。

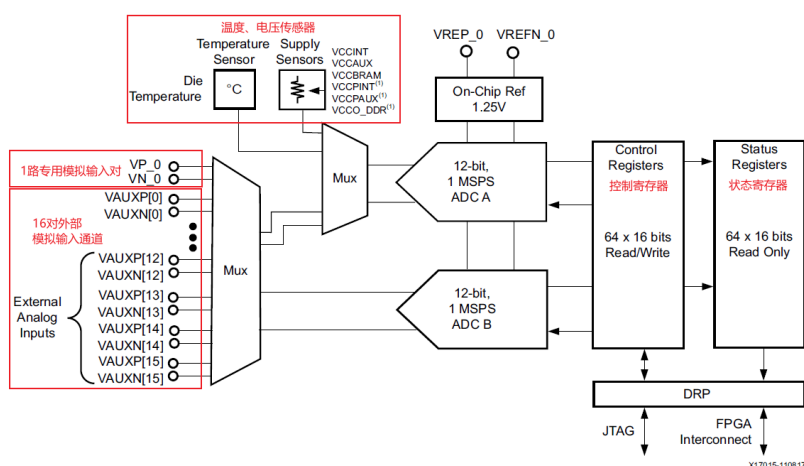


图 2-9 XDAC 原理

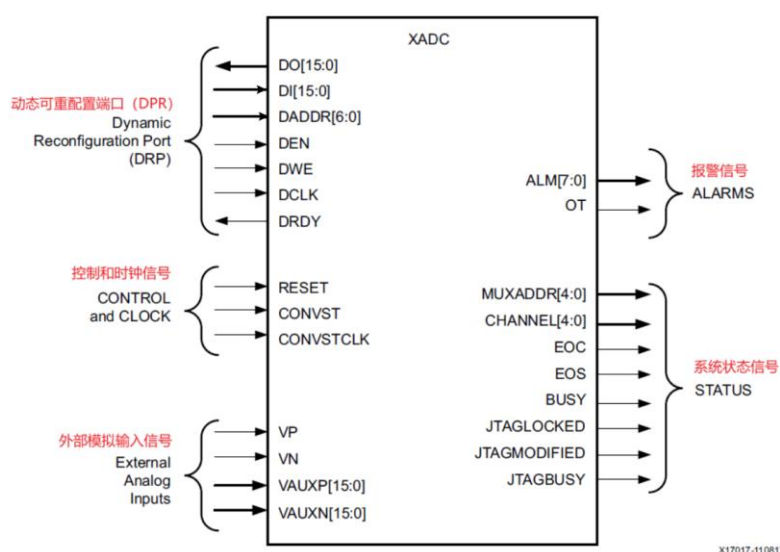


图 2-10 模数转换 XADC 模块框图

首先，通过 IP 目录添加 XADC IP 核到设计中，通过 GUI 对 XADC Wizard IP 核模块进行配置。根据用户设置，自动配置相关寄存器。然后，新建一个读取模块用于读取模拟信号转化的数字信号，在本项目中为 XADC 输出模块(data_output.v)。XADC 的模块结构如下图所示：

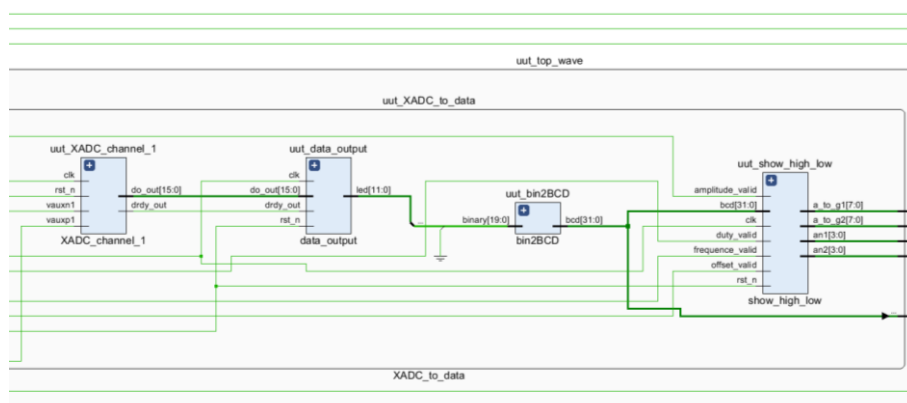


图 2-11 模数转换 XDAC 逻辑图

对于数码管输出模块(show_high_low.v)，需要将 XADC 转换后的 2 进制数字信号转换为 BCD 码，故需要加入一个码制转换模块(bin2BCD.v)，转换后的为 8 位 BCD 码，用于数码管的显示输出。

2.3.4 波形产生模块

利用 DDS 原理，产生波形，下面实例化了 3 个子模块。

方波模块(square.v)，正弦波模块(sine.v)，三角波模块(triangle.v)，根据输入的参数，以及 XADC 的调节参数，将产生波形。并根据需要的波形，进行数据的选择性输出(select_wave.v)。

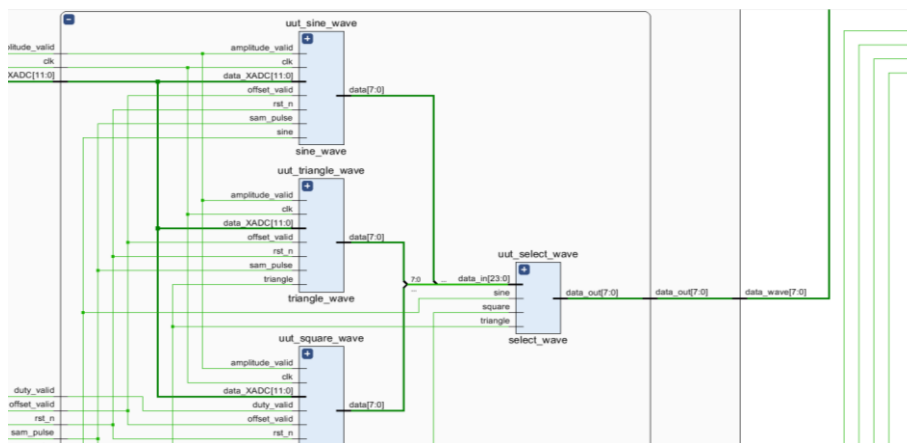


图 2-12 波形发生模块逻辑图

下面对于 DDS（直接数字合成）原理做一简单介绍：直接数字合成（Direct Digital Synthesis, DDS）是一种数字信号生成技术，广泛应用于波形发生器、频率合成器和信号处理系统中。DDS 能够生成高精度、可编程的波形，如正弦波、方波和三角波。其主要原理是通过数字处理和数模转换技术，以高精度和高稳定性生成各种波形信号。

（1）DDS 系统的核心组件包括：

1. 相位累加器（Phase Accumulator, PA）：相位累加器是 DDS 的核心，它通过不断累加相位增量（Phase Increment, PI）产生一个线性增长的相位信号。相位累加器的输出通常是一个固定位宽的数字信号，表示当前的相位。
2. 相位增量（Phase Increment, PI）：相位增量决定了输出信号的频率。相位增量的大小由控制信号设定，改变相位增量即可改变输出信号的频率。
3. 波形查找表（Waveform Lookup Table, LUT）：波形查找表存储了一周期波形的数字化样本（如正弦波的样本值）。相位累加器的输出作为查找表的地址，从查找表中读取相应的波形值。
4. 频率控制字（Frequency Control Word, FCW）：决定输出信号的频率，其值与相位增量有关。

（2）DDS 系统的基本工作流程：

相位累加：在每个时钟周期中，相位累加器累加相位增量，产生一个新的相位值。
 波形查找：将相位累加器的输出作为地址，查找波形查找表中对应的波形样本值。
 数模转换：将查找表输出的数字波形样本通过 DAC 转换为模拟信号，输出所需波形。

（3）频率调整

DDS 系统输出信号的频率由相位增量决定。相位增量的计算公式为：

$$\Delta\theta = \frac{f_{out} \times 2^{B_{\theta(n)}}}{f_{clk}}$$

其中：

$\Delta\theta$ 是相位增量。

f_{out} 是频率控制字。

f_{clk} 是系统时钟频率。

n 是相位累加器的位宽。

通过改变频率控制字 f_{out} 的值，可以调整输出信号的频率。较大的频率控制字对应较大的相位增量，从而产生较高频率的输出信号。

$$f_{out} = \frac{f_{clk} \times \Delta\theta}{2^{B_{\theta(n)}}}$$

具体的原理图，如下图所示：

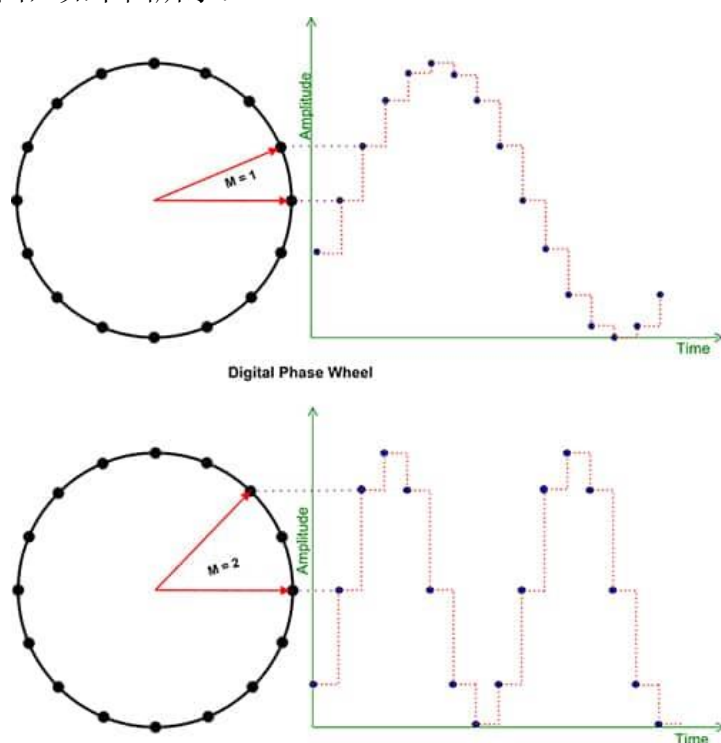


图 2-13 DDS 改变频率原理图

(4) 幅值调整

DDS 系统输出信号的幅值可以通过以下几种方式调整：

查找表调整：直接调整波形查找表中的样本值。增加查找表中的样本值可以提高输出信号的幅值。

数模转换器调整：在数模转换器之后，使用可变增益放大器调整模拟信号的幅值。

数字乘法器调整：在查找表输出和数模转换器之间使用数字乘法器，按比例缩放查找表输出的样本值。

2.4 硬件占用

由下图可见本实验的硬件占用情况：

由 Implementation report 可见硬件占用情况较好，查找表 LUT 占用了 25%，触发器 FF 1%，DSP 占用 29%，IO 占用情况为 15%。

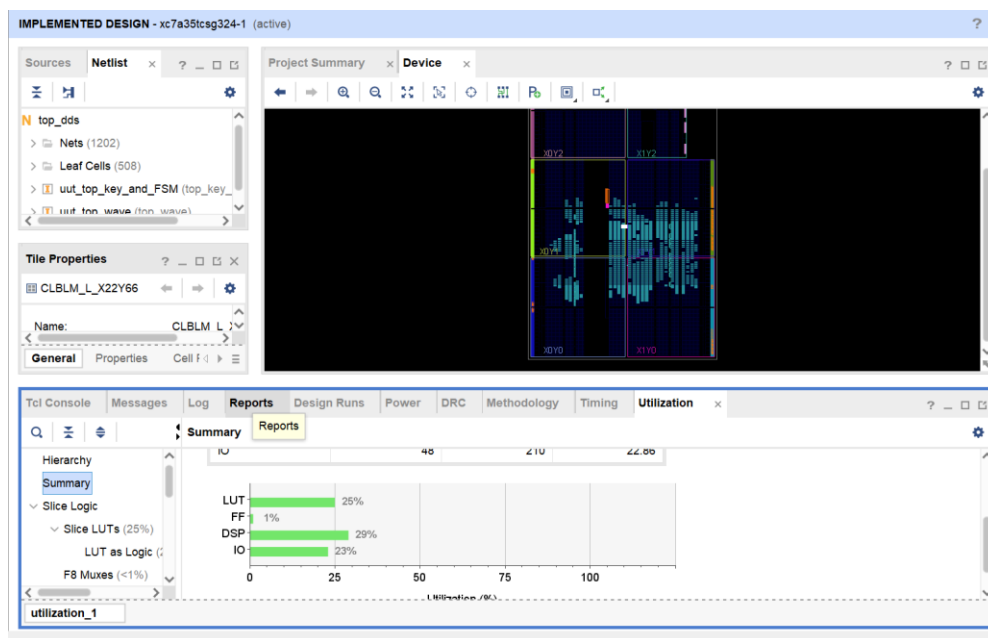


图 2-14 Implementation report

综上所述，该实验的占用情况较好，说明代码逻辑较为清晰，所占用的硬件较少，但仍有优化的空间。

3 系统调试

3.1 遇到的问题

3.1.1 系统出现问题排查错误

在编写和调试 FPGA 代码的过程中，经常会遇到一些小错误，这些错误可能来源于语法、逻辑设计、信号时序等多个方面。系统出现问题时，排查错误是一个挑战。主要困难包括：

复杂的代码结构：随着设计的复杂度增加，代码的模块化和层次化导致错误排查变得困难，需要逐层检查每个模块的功能和连接。

时序问题：FPGA 设计中时钟域跨越、信号延迟等时序问题较为隐蔽，难以通过简单的代码审查发现，必须通过时序分析工具进行检测。

硬件依赖：某些错误只有在硬件上运行时才会显现出来，难以在仿真环境中复现和排查，需要实际硬件调试工具的辅助。

按键个数的选择问题(选择 5 个按键，还是两个按键)

3.1.2 修改不同参数时，不同参数的相互影响问题

在调节波形发生器的参数时，不同参数之间存在相互影响，这对系统的设计和实现提出了挑战：

如果只采用两个按键，不同参数调节相互影响：

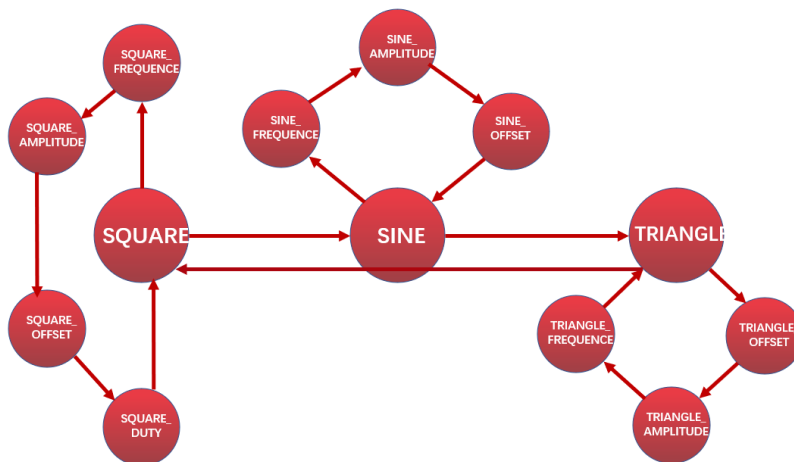


图 2-14 采用 2 个按键时不同参数相互影响

除此之外，还有：

直流偏移与幅值的耦合：调节直流偏移会改变波形的基准电平，从而影响幅值的实际表现。例如，在高直流偏移情况下，波形的有效幅值范围会受到限制，可能导致

波形失真或幅值不足。

占空比与频率的耦合：对于方波，占空比调节和频率调节之间也存在相互影响。例如，在高频率下，占空比的调节精度会受到时钟周期的限制，难以实现精确的占空比控制。

XADC 采样速率与参数调节的平滑性：通过 XADC 调节参数时，采样速率和参数变化的平滑性之间存在权衡。如果采样速率过低，参数变化会显得不连续；如果采样速率过高，可能会引入噪声和不稳定性。

3.1.3 基于 DDS 波形发生原理无法改变方波占空比

DDS（直接数字合成）波形发生器在生成正弦波和三角波时表现良好，但在生成方波并需要调节其占空比时，遇到了一些困难。

相位累加器的局限：DDS 利用相位累加器生成波形，相位累加器适用于周期性波形的生成，但无法直接控制方波的高低电平持续时间，导致占空比无法灵活调整。

波形查找表的局限：波形查找表存储的是一周期内波形的离散样本，对于固定占空比的方波，这种方法有效，但无法满足可变占空比的需求，需要额外的逻辑来动态调整方波的高低电平比例。

3.1.4 芯片 DAC0832 只输出高电平，无法进行电压转换

在使用 DAC0832 进行数模转换时，发现其只能输出高电平，无法正确进行电压转换。

电源和接地连接错误：如果电源和地连接不正确，DAC0832 无法正常工作，可能只输出高电平或无输出。

控制信号配置不当：DAC0832 需要正确的控制信号（如 CS、WR 等）才能正常工作，如果这些信号配置错误，DAC 可能无法接收和处理输入数据。

3.1.5 数码管不亮无输出

同一个输出，能否约束到两个引脚上？

在连接数码管后发现其不亮，没有输出显示，这可能由多个原因引起。

电源供电问题：数码管需要正确的电源供电，如果电源未正确连接或电压不稳定，数码管将无法正常工作。

驱动信号问题：数码管的段选信号和位选信号需要正确驱动，错误的驱动信号或电平可能导致数码管无法点亮。

引脚复用问题：如果数码管的输出引脚被用于其他功能，引脚复用配置不当可能导致信号冲突，影响数码管显示。

3.2 解决方案

3.2.1 设置多层模块分别仿真、调试

由为了提高系统的可靠性和便于错误排查，采用了多层模块设计和分层仿真调试的方法。系统由顶层到底层共设计了 4 层 20 个模块，并编写了 10 个仿真程序。这样可以分别对各个模块进行独立的仿真和调试，确保每个模块功能的正确性。具体步骤包括：

模块化设计：将系统功能划分为多个独立的模块，每个模块实现单一功能，便于调试和维护。

分层仿真：从底层模块开始逐层进行仿真和测试，确保每一层模块的正确性，再进行下一层的仿真。

综合调试：在顶层模块整合后，进行整体仿真和调试，确保各个模块之间的接口和协作正确无误。

3.2.2 选择 5 个按键代替原来的 2 个按键

原来选择 2 个按键，一个控制波形 `key_wave`，一个控制参数使能信号 `key_modify`。为了避免参数相互影响，原来使用的 2 个按键被改进为 5 个按键。具体分配如下：

key_wave：用于改变波形类型的按键。

key_frequency：用于改变波形频率的按键。

key_amplitude：用于改变波形幅值的按键。

key_offset：用于改变波形直流偏移信号的按键。

key_duty：用于改变方波占空比的按键。

选择 5 个按键可以更清晰地分开不同功能的控制，避免了参数之间的互相影响。同时，选择合适的参数调节范围和适当的采样脉冲数量，以确保每个参数在调节时都能得到准确的控制和良好的用户体验。

3.2.3 使用采样脉冲，改变方波占空比

计为了实现方波占空比的调节，采用了采样脉冲的方法。具体实现步骤如下：

周期分段：将一个波形周期分为 255 个采样点，这样可以在一个周期内发出 255 个采样脉冲。

采样控制：在每一个采样脉冲改变后，保持一定的时间，确保数字信号能稳定转换为模拟信号。

电平翻转：根据设定的占空比，在固定的采样点实现电平翻转，从而控制方波的占空比。

这种方法能够精确地控制方波的高低电平时间，使得占空比可以灵活调节，满足不同应用的需求。

3.2.4 必须要对于控制信号在约束文件中定义

在 FPGA 设计中，控制信号的定义不仅需要在顶层文件中明确，还必须在约束文件中进行定义。如果只在顶层文件中定义控制引脚，可能会导致信号无法正确输出。具体步骤包括：

顶层文件定义：在顶层文件中定义所有控制信号的引脚。

约束文件配置：在约束文件中对这些引脚进行详细的约束配置，确保信号能够正确映射到 FPGA 的物理引脚上。

验证输出：通过仿真和实际测试，验证控制信号是否能够正确输出并发挥作用。

3.2.5 数码管的输出信号无法复用

在设计中发现数码管的输出信号无法同时连接到两个引脚上。虽然在代码中没有报错，但实际测试中发现这样会导致数码管无法正常工作。解决方法包括：

独立驱动：确保每个数码管的驱动信号都是独立的，不与其他信号复用。

信号隔离：如果需要多个信号控制数码管，使用信号隔离或复用器来切换控制信号，确保每次只有一个有效信号控制数码管。

调试验证：通过仿真和实际测试，验证数码管的驱动信号是否正确，确保数码管能够正常显示预期的输出。

3.3 创新设计

3.3.1 采用 XADC 进行参数的输入

为了提高系统的灵活性和用户体验，本设计采用了 XADC（可扩展模数转换器）进行参数的输入。XADC 模块的创新点在于：

高精度参数调节：通过外部接口输入模拟信号，XADC 模块将其转换为数字信号，实现高精度的参数调节。

实时性：XADC 能够实现实时的参数输入和调节，用户可以快速、连续地调整波形的频率、幅值、直流偏移和占空比等参数。

灵活性：利用 XADC 模块，用户可以方便地通过外部硬件接口对波形生成器进行控制，增强了系统的灵活性和可操作性。

3.3.2 数码管显示时设计了修改参数时数码管闪动

在数码管显示设计中，增加了参数修改时数码管闪动的功能，具体设计如下：

闪动提示：当用户修改参数时，相关的数码管会闪动提示，这样可以明确告诉用户当前正在修改的参数，避免误操作。

实时反馈：通过数码管闪动的实时反馈机制，用户可以更直观地了解当前参数的状态和变化，提高操作的便捷性和准确性。

可视化效果：闪动设计不仅增加了系统的可视化效果，还提升了用户交互体验，使操作更加直观和人性化。

3.3.3 利用 LED 灯与数码管的结合显示波形信息

为了更直观地显示波形信息，本设计将 LED 灯与数码管结合起来，实现了更丰富的显示功能：

波形类型显示：通过不同颜色和状态的 LED 灯显示当前的波形类型（如方波、正弦波、三角波），用户可以一目了然地了解当前波形。

参数信息显示：数码管用于显示具体的波形参数（如频率、幅值、直流偏移和占空比等），LED 灯用于指示当前显示的参数类型。

动态显示：结合数码管和 LED 灯的动态显示效果，在参数修改和波形切换时提供实时反馈，增强用户体验。例如，当用户按下修改按钮时，相关的数码管和 LED 灯会同步闪动或改变状态，提示当前正在进行的操作。

4 系统使用说明

4.1 数码管

为了更直观、全面地显示波形信息，本设计采用了 8 个 7 段数码管和 8 个 LED 灯的结合来显示波形参数、单位以及波形类型。具体设计如下：

4.1.1 数码管显示

波形类型（数码管 1-2）：使用简写字符表示当前的波形类型，例如：Sq 表示方波（Square Wave）。Si 表示正弦波（Sine Wave）。Tr 表示三角波（Triangle Wave）。

波形参数数值（数码管 3-6）：数码管 3，千位；数码管 4，百位；数码管 5，十位；数码管 6，个位。这四个数码管用于显示波形参数的具体数值，例如频率、幅值、直流偏移量等。

单位（数码管 7-8）：通过不同的字符表示当前波形参数的单位，例如：H 表示赫兹（Hz），用于频率 V 表示伏特（V），用于幅值。D 表示直流偏移（DC Offset）。% 表示占空比（Duty Cycle）。

4.1.2 LED 灯显示

波形参数指示（LED1-4）：LED1，频率（Frequency）；LED2，幅值（Amplitude）；LED3，直流偏移（DC Offset）；LED4，占空比（Duty Cycle）。当某个参数正在被调整时，对应的 LED 灯会亮起或闪烁，以提示用户当前正在调节的参数类型。

工作状态指示（LED5-6）：LED5，运行状态（Running）；LED6，停止状态（Stopped）。这两个 LED 灯用于指示波形发生器的工作状态，运行时 LED5 亮，停止时 LED6 亮。

通过以上设计，用户可以直观地看到当前波形的频率、波形类型以及相关单位信息，同时通过 LED 灯清楚地知道当前显示和调节的参数类型及系统工作状态。这样，不仅提高了信息显示的丰富性和直观性，还增强了用户的交互体验和操作的便捷性。

4.2 按键控制

4.2.1 按键(5 个)控制

在 DDS 波形发生器中，每个按键按下后都触发了不同的功能，波形类型切换按键（key_wave）：

用户按下此按键后，系统会循环切换波形类型。每次按下，当前波形类型会切换到下一个类型，包括方波、正弦波和三角波。这种循环切换的设计使用户能够轻松地选择所需的波形类型，方便实用。

4.2.2 按键与电位器配合

(1) 频率调节按键 (key_frequency):

按住此按键并旋转电位器，用户可以逐步调节波形的频率。顺时针旋转电位器增加频率，逆时针旋转则减小频率。这种按键设计使用户能够精确地调节波形的频率，满足不同应用场景的需求。

(2) 幅值调节按键 (key_amplitude):

用户按下此按键并旋转电位器时，可以调节波形的幅值。顺时针旋转电位器增加幅值，逆时针旋转减小幅值。这种操作使用户能够轻松地调整波形的振幅，以适应不同的信号处理需求。

(3) 直流偏移调节按键 (key_offset):

按下此按键并旋转电位器时，用户可以调节波形的直流偏移量。顺时针旋转电位器会使波形向上移动，逆时针旋转则使波形向下移动。这样的设计使用户可以轻松地调整波形的垂直位置，以满足不同的应用要求。

(4) 方波占空比调节按键 (key_duty):

仅对方波有效。按下此按键并旋转电位器时，用户可以调节方波的占空比，即高电平与周期的比值。顺时针旋转电位器增加占空比，逆时针旋转则减小占空比。这种设计使用户能够轻松地调整方波的占空比，以实现更灵活的波形控制。

通过以上按键功能的详细描述，用户能够清楚地了解每个按键按下后的具体功能，从而更加方便地操作 DDS 波形发生器，实现所需的波形生成和调节。

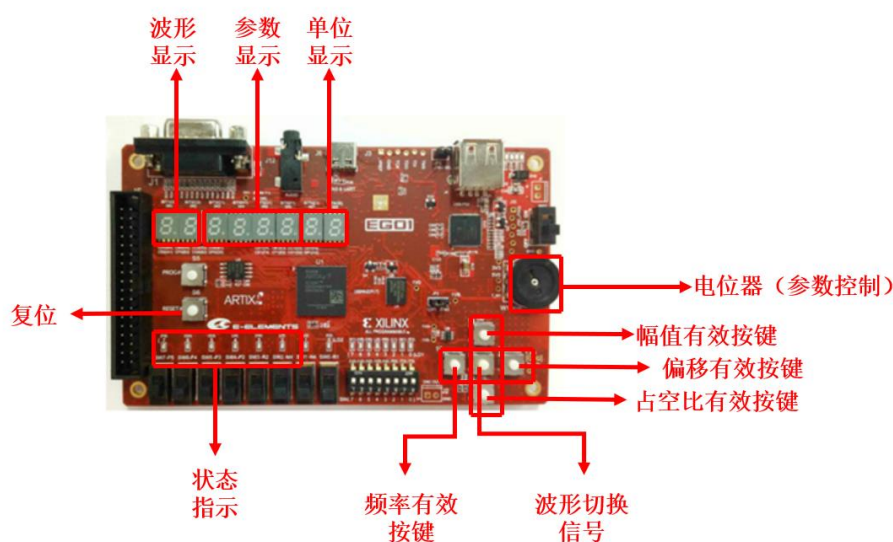


图 4.1 DDS 波形发生器功能按键示意图

5 参考文献

- [1]余勇,郑小林.基于 FPGA 的 DDS 正弦信号发生器的设计和实现[J].电子器件,2005,(03):596-599.
- [2]潘志浪.基于 FPGA 的 DDS 信号源的设计[D].武汉理工大学,2007.
- [3]薛文.DDS 任意波形发生器的设计与实现[D].南京理工大学,2004.
- [4]孙昊.基于 FPGA 的 DDS 信号源设计[D].电子科技大学,2009.
- [5]高泽溪,高成.直接数字频率合成器(DDS)及其性能分析[J].北京航空航天大学学报,1998,(05):121-124.

6 总结

总结部分包括课程收获，心得体会，对综合实验的总结以及意见建议四个部分，主要写的是我对于本课程以及实验的一些感想。

6.1 课程收获

经过 12 周的学习，我们终于结束了《数字电子技术》这门课的学习，并且在学习的过程中，我们又完成了《数字电子技术》的相关实验及其相关课程。

通过学习《数字电子技术》这门课，我学到了很多，不仅学到了充实的理论知识，为我之后进一步的专业课的学习奠定了基础，更掌握了 Verilog HDL 这门硬件编程语言，为我们以后的科研和学习提供了一种新的工具。

在实验过程中，我们需要将基础知识与硬件编程结合起来，在这个过程中，我们进一步提升了对于基本数字电子技术知识的理解，同时，还需要查阅用户手册、数据手册，在这个过程中也极大地提高了我的自学能力。

6.2 心得体会

在进行实验的过程中，我发现硬件编程与软件编程有很大的区别：

首先，我们需要将程序中的变量与 FPGA 的引脚对应起来，在这个过程中，我必须学会查阅引脚定义然后将其与变量关联；

其次，对于 Verilog HDL 语言，在完成源文件的编程后，还可以通过仿真程序来验证程序逻辑的正确性，通过波形，我们可以观察到不同情况下输入输出的高低电平情况；

最后，在对 FPGA 编程的过程中，我明白了硬件编程不仅要能完成需求，更要考虑到芯片的占用情况，考虑程序运行的时间以及程序的可读性和可修改性等等各项因素。仅仅只是完成了功能是远远不够的，还需要尽量简化程序的逻辑，以提高运行的效率。

6.3 实验总结

对于在本次实验中，我们成功设计并实现了基于 FPGA 的 DDS 波形发生器。通过对 DDS 原理的理解和实践，我们完成了具有方波、正弦波和三角波等多种波形类型的生成，同时实现了频率、幅值、直流偏移和方波占空比等参数的调节功能。在实验过程中，我们遇到了一些挑战和困难，但通过不断努力和解决问题，最终取得了令人满意的成果。

首先，我们遇到了系统出现问题时的排查与错误解决。通过设置多层模块并编写多个仿真程序，我们能够逐层分析和调试，快速定位问题并采取相应的解决措施。其次，在按键个数的选择问题上，我们经过思考和讨论，最终选择了 5 个按键以提供更多的操作选项和更丰富的用户体验。此外，基于 DDS 波形发生原理，我们发现无法直接改变方波的占空比，但通过采样脉冲的方式实现了方波占空比的调节。对于芯片 DAC0832 的输出问题，我们将控制信号在约束文件中定义，解决了输出问题。最后，数码管不亮无输出的问题得到解决，我们发现数码管的输出信号无法复用，于是对信号进行了约束，成功解决了问题。

通过实验，我们不仅掌握了 FPGA 的开发流程和基本原理，还深入了解了 DDS 波形发生器的工作原理和实现方法。我们通过合理的设计和精心的调试，成功实现了一个功能完善、稳定可靠的 DDS 波形发生器，为进一步深入研究和应用提供了有力支持。在今后的学习和工作中，我将继续积极探索和应用 FPGA 技术，不断提升自己的技能水平，为科技进步和社会发展做出更大的贡献。

7 附 录

附录部分为源程序，包括各个.v 文件，要有注释，（以下重复性的端口或模块，只注释一次）。

7.1 源程序

7.1.1 顶层模块(top_dds.v)

```
`timescale 1ns / 1ps
module top_dds(
    input clk, //时钟信号
    input rst_n, //复位信号
    input vauxp1, //模拟输入信号
    input vauxn1, //模拟输入信号
    input ILE, //DAC0832 使能控制端，高有效
    input CS_n, //DAC0832 使能控制端，低有效
    input WR1_n, //DAC0832 使能控制端，低有效
    input WR2_n, //DAC0832 使能控制端，低有效
    input XFER_n, //DAC0832 使能控制端，低有效
    input key_wave, //波形转换输入按键
    input key_frequence, //频率有效控制输入按键
    input key_amplitude, //幅值有效控制输入按键
    input key_offset, //偏移有效控制输入按键
    input key_duty, //占空比有效控制输入信号
    output [3:0] an1, //数码管 1 位选信号
    output [3:0] an2, //数码管 2 位选信号
    output [7:0] a_to_g1, //数码管 1 管段信号
    output [7:0] a_to_g2, //数码管 2 管段信号
    output wire square, //输出波形状态信号
    output wire sine, //输出波形状态信号
    output wire triangle, //输出波形状态信号
    output wire frequence_valid, //频率有效控制信号
    output wire offset_valid, //偏移有效控制信号
    output wire amplitude_valid, //幅值有效控制信号
    output wire duty_valid, //占空比有效控制信号
```

```

output [7:0] data_wave //模数转换的数字信号
);
wire [11:0] data; //XADC 转换后的 2 进制信号
//顶层波形模块(top_wave.v)
top_wave uut_top_wave(
.clk(clk),
.rst_n(rst_n),
.vauxp1(vauxp1),
.vauxn1(vauxn1),
.ILE(ILE),
.CS_n(CS_n),
.WR1_n(WR1_n),
.WR2_n(WR2_n),
.XFER_n(XFER_n),
.square(square),
.sine(sine),
.triangle(triangle),
.frequency_valid(frequency_valid),
.offset_valid(offset_valid),
.amplitude_valid(amplitude_valid),
.duty_valid(duty_valid),
.data(data),
.an1(an1),
.an2(an2),
.a_to_g1(a_to_g1),
.a_to_g2(a_to_g2),
.data_wave(data_wave)
);
//按键与状态机模块(top_key_and_FSM.v)
top_key_and_FSM uut_top_key_and_FSM(
.clk(clk),
.rst_n(rst_n),
.key_wave(key_wave),
.key_frequency(key_frequency),
.key_amplitude(key_amplitude),
.key_offset(key_offset),
.key_duty(key_duty),
.square(square),
.sine(sine),

```



```

        .triangle(triangle),
        .frequence(frequence_valid),
        .offset(offset_valid),
        .amplitude(amplitude_valid),
        .duty(duty_valid)
    );
endmodule

7.1.2 按键与状态机模块(top_key_and_FSM.v)
`timescale 1ns / 1ps
module top_key_and_FSM(
    input clk,
    input rst_n,
    input key_wave,
    input key_frequence,
    input key_amplitude,
    input key_offset,
    input key_duty,
    output square,
    output sine,
    output triangle,
    output frequency, //频率有效控制信号
    output offset, //偏移有效控制信号
    output amplitude, //频率有效控制信号
    output duty//频率有效控制信号
);
    wire get_key_wave; //波形按键上升沿有效信号
    wire get_key_frequence; //频率按键上升沿有效信号
    wire get_key_amplitude; //幅值按键上升沿有效信号
    wire get_key_offset; //直流偏移按键上升沿有效信号
    wire get_key_duty; //占空比按键上升沿有效信号
//5 个按键综合模块(get_5key.v)
    get_5key uut_get_5key(
        .clk(clk),
        .rst_n(rst_n),
        .key_wave(key_wave),
        .key_frequence(key_frequence),
        .key_amplitude(key_amplitude),
        .key_offset(key_offset),
        .key_duty(key_duty),

```

```
.get_key_wave(get_key_wave),
.get_key_frequency(get_key_frequency),
.get_key_amplitude(get_key_amplitude),
.get_key_offset(get_key_offset),
.get_key_duty(get_key_duty)
);
//有限状态机模块(FSM_5key.v)
FSM_5key uut_FSM_5key(
.clk(clk),
.rst_n(rst_n),
.get_key_wave(get_key_wave),
.get_key_frequency(get_key_frequency),
.get_key_amplitude(get_key_amplitude),
.get_key_offset(get_key_offset),
.get_key_duty(get_key_duty),
.square(square),
.sine(sine),
.triangle(triangle),
.frequency(frequency),
.offset(offset),
.amplitude(amplitude),
.duty(duty)
);
endmodule
```

7.1.3 按键 5 个的综合模块(get_5key.v)

```
`timescale 1ns / 1ps
module get_5key(
    input clk,
    input rst_n,
    input key_wave,
    input key_frequency,
    input key_amplitude,
    input key_offset,
    input key_duty,
    output wire get_key_wave,
    output wire get_key_frequency,
    output wire get_key_amplitude,
    output wire get_key_offset,
    output wire get_key_duty
);
```

```
//单个的按键消抖模块(key_debounce.v)
```

```
//波形输入按键消抖
```

```
key_debounce u_key_wave(
    .clk(clk),
    .rst_n(rst_n),
    .key(key_wave),
    .key_p_flag(get_key_wave)
);
```

```
//频率改变有效输入按键消抖
```

```
key_debounce u_key_frequence(
    .clk(clk),
    .rst_n(rst_n),
    .key(key_frequence),
    .key_p_flag(get_key_frequence)
);
```

```
//幅值改变有效输入按键消抖
```

```
key_debounce u_key_amplitude(
    .clk(clk),
    .rst_n(rst_n),
    .key(key_amplitude),
    .key_p_flag(get_key_amplitude)
);
```

```
//偏移改变有效输入按键消抖
```

```
key_debounce u_key_offset(
    .clk(clk),
    .rst_n(rst_n),
    .key(key_offset),
    .key_p_flag(get_key_offset)
);
```

```
//占空比改变有效输入按键消抖
```

```
key_debounce u_key_duty(
    .clk(clk),
    .rst_n(rst_n),
    .key(key_duty),
    .key_p_flag(get_key_duty)
);
```

```
endmodule
```

7.1.4 按键消抖模块(key_debounce.v)

```
`timescale 1ns / 1ps
```

```
module key_debounce(
```

```
input clk,
input rst_n,
input key, //输入按键信号
output reg key_p_flag //输出按键上升沿有效信号
);
reg key_reg; //按键输入寄存器
reg key_value; //消抖后按键
reg [20:0]delay_cnt = 0; //延时计数器
reg key_n_flag;
parameter DELAY_YIME = 21'd15_00_000; //15ms 延时时间
//key_reg: 按键输入寄存器
always@(posedge clk,negedge rst_n)
    if (!rst_n)
        key_reg <= 1'b0;
    else
        key_reg <= key;
//delay_cnt:延时计数器
always@(posedge clk,negedge rst_n)
    if (!rst_n)
        delay_cnt <= 21'd0;
    else if (key != key_reg)
        delay_cnt <= DELAY_YIME;
    else if (delay_cnt>0)
        delay_cnt <= delay_cnt-1'b1;
    else
        delay_cnt <= 21'd0;
//消抖后按键
always@(posedge clk,negedge rst_n)
    if (!rst_n)
        key_value <= 1'b0;
    else if (delay_cnt == 1'b1)
        key_value <= key;
//key_p_flag: 按键上升沿标志信号
always@(posedge clk,negedge rst_n)
    if (!rst_n)
        key_p_flag <= 1'b0;
    else if (delay_cnt == 1'b1&&key == 1)
        key_p_flag <= 1'b1;
    else
        key_p_flag <= 1'b0;
```

//key_n_flag: 按键下降沿标志信号

```
always@(posedge clk,negedge rst_n)
    if (!rst_n)
        key_n_flag <= 1'b0;
    else if (delay_cnt == 1'b1 && key == 0)
        key_n_flag <= 1'b1;
    else
        key_n_flag <= 1'b0;
endmodule
```

7.1.5 有限状态机模块(FSM_5key.v)

```
`timescale 1ns / 1ps
```

```
module FSM_5key(
    input clk,
    input rst_n,
    input get_key_wave,
    input get_key_frequence,
    input get_key_amplitude,
    input get_key_offset,
    input get_key_duty,
    output reg square,
    output reg sine,
    output reg triangle,
    output reg frequency,
    output reg offset,
    output reg amplitude,
    output reg duty
);
```

//状态编码

```
parameter SQUARE = 4'b0000; //方波输出状态
parameter SQUARE_FREQUENCY = 4'b0001; //方波频率可变状态
parameter SQUARE_AMPLITUDE = 4'b0010; //方波幅值可变状态
parameter SQUARE_OFFSET = 4'b0011; //方波直流偏移可变状态
parameter SQUARE_DUTY = 4'b0100; //方波占空比可变状态
parameter SINE = 4'b0101; //正弦波输出状态
parameter SINE_FREQUENCY = 4'b0110; //正弦波频率可变状态
parameter SINE_AMPLITUDE = 4'b0111; //正弦波幅值可变状态
parameter SINE_OFFSET = 4'b1000; //正弦波直流偏移可变状态
parameter TRIANGLE = 4'b1001; //三角波输出状态
parameter TRIANGLE_FREQUENCY = 4'b1010; //三角波频率可变状态
```

```

parameter TRIANGLE_AMPLITUDE = 4'b1011; //三角波幅值可变状态
parameter TRIANGLE_OFFSET = 4'b1100; //三角波直流偏移可变状态
reg [3:0] cstate    = 4'b0000; //现态寄存器
reg [3:0] nstate    = 4'b0000; //次态寄存器
//使用三段式状态机描述方法
//cstate:第一段描述现态
always@(posedge clk,negedge rst_n)
    if (!rst_n) cstate <= SQUARE;
    else cstate <= nstate;
//nstate:第二段描述次态
always@(*)
    if (!rst_n)
        nstate = SQUARE;
    else
        case (cstate)
            //方波状态
            SQUARE:
                if (get_key_wave == 1) nstate = SINE;
                else if (get_key_frequence == 1) nstate = SQUARE_FREQUENCE;
                else if (get_key_amplitude == 1) nstate = SQUARE_AMPLITUDE;
                else if (get_key_offset == 1) nstate = SQUARE_OFFSET;
                else if (get_key_duty == 1) nstate = SQUARE_DUTY;
                else nstate = SQUARE;
            SQUARE_FREQUENCE:
                if (get_key_frequence == 1) nstate = SQUARE;
                else nstate = SQUARE_FREQUENCE;
            SQUARE_AMPLITUDE:
                if (get_key_amplitude == 1) nstate = SQUARE;
                else nstate = SQUARE_AMPLITUDE;
            SQUARE_OFFSET:
                if (get_key_offset == 1) nstate = SQUARE;
                else nstate = SQUARE_OFFSET;
            SQUARE_DUTY:
                if (get_key_duty == 1) nstate = SQUARE;
                else nstate = SQUARE_DUTY;
            //正弦波状态
            SINE:
                if (get_key_wave == 1) nstate = TRIANGLE;
                else if (get_key_frequence == 1) nstate = SINE_FREQUENCE;
                else if (get_key_amplitude == 1) nstate = SINE_AMPLITUDE;

```

```

else if (get_key_offset == 1) nstate = SINE_OFFSET;
else nstate = SINE;
SINE_FREQUENCY:
if (get_key_frequency == 1) nstate = SINE;
else nstate = SINE_FREQUENCY;
SINE_AMPLITUDE:
if (get_key_amplitude == 1) nstate = SINE;
else nstate = SINE_AMPLITUDE;
SINE_OFFSET:
if (get_key_offset == 1) nstate = SINE;
else nstate = SINE_OFFSET;

//三角波状态
TRIANGLE:
if (get_key_wave == 1) nstate = SQUARE;
else if (get_key_frequency == 1) nstate = TRIANGLE_FREQUENCY;
else if (get_key_amplitude == 1) nstate = TRIANGLE_AMPLITUDE;
else if (get_key_offset == 1) nstate = TRIANGLE_OFFSET;
else nstate = TRIANGLE;
TRIANGLE_FREQUENCY:
if (get_key_frequency == 1) nstate = TRIANGLE;
else nstate = TRIANGLE_FREQUENCY;
TRIANGLE_AMPLITUDE:
if (get_key_amplitude == 1) nstate = TRIANGLE;
else nstate = TRIANGLE_AMPLITUDE;
TRIANGLE_OFFSET:
if (get_key_offset == 1) nstate = TRIANGLE;
else nstate = TRIANGLE_OFFSET;
default: nstate = SQUARE;
endcase

//第三段：描述输出
always@(posedge clk, negedge rst_n)
if (!rst_n)
begin
square      <= 1'b1;
sine        <= 1'b0;
triangle    <= 1'b0;
frequency   <= 1'b0;
offset      <= 1'b0;
amplitude   <= 1'b0;
duty        <= 1'b0;
end
end

```

```

else
    case(nstate)
        SQUARE:
begin square <= 1'b1;sine <= 1'b0;triangle <= 1'b0;
frequency <= 1'b0;offset <= 1'b0;amplitude <= 1'b0; duty <= 1'b0; end
        SQUARE_FREQUENCY:
begin square <= 1'b1;sine <= 1'b0;triangle <= 1'b0;
frequency <= 1'b1;offset <= 1'b0;amplitude <= 1'b0; duty <= 1'b0; end
        SQUARE_OFFSET:
begin square <= 1'b1;sine <= 1'b0;triangle <= 1'b0;
frequency <= 1'b0;offset <= 1'b1;amplitude <= 1'b0; duty <= 1'b0; end
        SQUARE_AMPLITUDE:
begin square <= 1'b1;sine <= 1'b0;triangle <= 1'b0;
frequency <= 1'b0;offset <= 1'b0;amplitude <= 1'b1; duty <= 1'b0; end
        SQUARE_DUTY:
begin square <= 1'b1;sine <= 1'b0;triangle <= 1'b0;
frequency <= 1'b0;offset <= 1'b0;amplitude <= 1'b0; duty <= 1'b1; end
        SINE:
begin square <= 1'b0;sine <= 1'b1;triangle <= 1'b0;
frequency <= 1'b0;offset <= 1'b0;amplitude <= 1'b0; duty <= 1'b0; end
        SINE_FREQUENCY:
begin square <= 1'b0;sine <= 1'b1;triangle <= 1'b0;
frequency <= 1'b1;offset <= 1'b0;amplitude <= 1'b0; duty <= 1'b0; end
        SINE_OFFSET:
begin square <= 1'b0;sine <= 1'b1;triangle <= 1'b0;
frequency <= 1'b0;offset <= 1'b1;amplitude <= 1'b0; duty <= 1'b0; end
        SINE_AMPLITUDE:
begin square <= 1'b0;sine <= 1'b1;triangle <= 1'b0;
frequency <= 1'b0;offset <= 1'b0;amplitude <= 1'b1; duty <= 1'b0; end
        TRIANGLE:
begin square <= 1'b0;sine <= 1'b0;triangle <= 1'b1;
frequency <= 1'b0;offset <= 1'b0;amplitude <= 1'b0; duty <= 1'b0; end
        TRIANGLE_FREQUENCY:
begin square <= 1'b0;sine <= 1'b0;triangle <= 1'b1;
frequency <= 1'b1;offset <= 1'b0;amplitude <= 1'b0; duty <= 1'b0; end
        TRIANGLE_OFFSET:
begin square <= 1'b0;sine <= 1'b0;triangle <= 1'b1;
frequency <= 1'b0;offset <= 1'b1;amplitude <= 1'b0; duty <= 1'b0; end
        TRIANGLE_AMPLITUDE:
begin square <= 1'b0;sine <= 1'b0;triangle <= 1'b1;
frequency <= 1'b0;offset <= 1'b0;amplitude <= 1'b1; duty <= 1'b0; end
    
```



```

        default:
begin square <= 1'b1;sine <= 1'b0;triangle <= 1'b0;
frequence <= 1'b0;offset <= 1'b0;amplitude <= 1'b0; duty <= 1'b0; end
        endcase
endmodule

```

7.1.6 顶层波形模块(top_wave.v)

```

`timescale 1ns / 1ps
module top_wave(
    input clk,
    input rst_n,
    input vauxp1,
    input vauxn1,
    input ILE,
    input CS_n,
    input WR1_n,
    input WR2_n,
    input XFER_n,
    input square,
    input sine,
    input triangle,
    input frequence_valid,
    input offset_valid,
    input amplitude_valid,
    input duty_valid,
    output [11:0] data, //XADC 转换后的 2 进制数字信号
    output [3:0] an1,
    output [3:0] an2,
    output [7:0] a_to_g1,
    output [7:0] a_to_g2,
    output [7:0] data_wave
);
    wire sam_pulse; //DAC0832 的采样脉冲
    wire [13:0] frequence; //转换后的频率大小
    assign ILE = 1'b1; //DAC0832 控制信号上拉有效
    assign CS_n = 1'b0; //DAC0832 控制信号低有效
    assign WR1_n = 1'b0; //DAC0832 控制信号低有效
    assign WR2_n = 1'b0; //DAC0832 控制信号低有效
    assign XFER_n = 1'b0; //DAC0832 控制信号低有效
//XADC 模数转换模块(XADC_to_data.v)

```

```
XADC_to_data uut_XADC_to_data(
    .clk(clk),
    .rst_n(rst_n),
    .vauxp1(vauxp1),
    .vauxn1(vauxn1),
    .data(data),
    .an1(an1),
    .an2(an2),
    .a_to_g1(a_to_g1),
    .a_to_g2(a_to_g2),
    .frequence_valid(frequence_valid),
    .amplitude_valid(amplitude_valid),
    .offset_valid(offset_valid),
    .duty_valid(duty_valid)
);
//采样脉冲模块(pulse.v)
pulse uut_pulse(
    .clk(clk),
    .rst_n(rst_n),
    .data(data),
    .sam_pulse(sam_pulse),
    .frequence(frequence), //频率经过十进制数后的转换值
    .frequence_valid(frequence_valid)
);
//波形发生模块(wave.v)
wave uut_wave(
    .clk(clk),
    .rst_n(rst_n),
    .square(square),
    .sine(sine),
    .triangle(triangle),
    .sam_pulse(sam_pulse),
    .data_out(data_wave), //DAC0832 数模转换的数值
    .offset_valid(offset_valid),
    .amplitude_valid(amplitude_valid),
    .duty_valid(duty_valid),
    .data_XADC(data)
);
endmodule
```

7.1.7 模数转换(XADC_to_data.v)

```

`timescale 1ns / 1ps
module XADC_to_data(
    input clk,
    input rst_n,
    input vauxp1,
    input vauxn1,
    input frequency_valid,
    input amplitude_valid,
    input offset_valid,
    input duty_valid,
    output [11:0] data, //XADC 转换后的 2 进制数转换为 BCD 码前三位
    output [3:0] an1,
    output [3:0] an2,
    output [7:0] a_to_g1,
    output [7:0] a_to_g2
);
    wire [15:0] do_out; //模数转换结果前 12 位有效
    wire drdy_out; //XADC 转换完成标志
    wire [31:0] bcd; //将 XADC 转换后的 2 进制数转换为 BCD 码
    wire [11:0] led; //模数转换结果前 12 位
    assign data[11:0] = bcd[27:16];
    // 模数转换 XADC 的 IP 核模块(XADC_channel_1.v)
    XADC_channel_1 uut_XADC_channel_1(
        .clk(clk),
        .rst_n(rst_n),
        .vauxp1(vauxp1),
        .vauxn1(vauxn1),
        .do_out(do_out),
        .drdy_out(drdy_out)
    );
    //数据转换输出模块(data_output.v)
    data_output uut_data_output(
        .clk(clk),
        .rst_n(rst_n),
        .drdy_out(drdy_out),
        .do_out(do_out),
        .led(led)
    );
    //二进制转换为 BCD 码输出在数码管上(bin2BCD.v)

```

```

    bin2BCD uut_bin2BCD(
        .binary({8'd0,led}),
        .bcd(bcd)
    );
//数码管显示模块(show_high_low.v)
    show_high_low uut_show_high_low(
        .clk(clk),
        .rst_n(rst_n),
        .bcd(bcd),
        .frequence_valid(frequence_valid),
        .amplitude_valid(amplitude_valid),
        .offset_valid(offset_valid),
        .duty_valid(duty_valid),
        .an1(an1),
        .an2(an2),
        .a_to_g1(a_to_g1),
        .a_to_g2(a_to_g2)
    );

endmodule

```

7.1.8 模数转换 XADC 的 IP 核模块(XADC_channel_1.v)

```

`timescale 1ns / 1ps
module XADC_channel_1(
    input clk,
    input rst_n,
    input vauxp1,
    input vauxn1,
    output [15:0] do_out,
    output drdy_out
);
    wire eoc_out;
    wire reset_in;
    assign reset_in = !rst_n;//XADC 的 IP 核复位高有效
//XADC 的 IP 核
    xadc_wiz_0 u1 (
        .di_in(16'b0),                // input wire [15 : 0] di_in
        .daddr_in(7'h11),             // input wire [6 : 0] daddr_in
        .den_in(eoc_out),             // input wire den_in
        .dwe_in(1'b0),               // input wire dwe_in
        .drdy_out(drdy_out),          // output wire drdy_out
    );

```

```

.do_out(do_out),           // output wire [15 : 0] do_out
.dclk_in(clk),             // input wire dclk_in
.reset_in(reset_in),       // input wire reset_in
.vp_in(0),                 // input wire vp_in
.vn_in(0),                 // input wire vn_in
.vauxp1(vauxp1),           // input wire vauxp1
.vauxn1(vauxn1),           // input wire vauxn1
.channel_out(),            // output wire [4 : 0] channel_out
.eoc_out(eoc_out),         // output wire eoc_out
.alarm_out(),              // output wire alarm_out
.eos_out(),                // output wire eos_out
.busy_out()                // output wire busy_out
);
endmodule

```

7.1.9 数据转换输出模块(data_output.v)

```

`timescale 1ns / 1ps
module data_output(
    input clk,
    input rst_n,
    input drdy_out,
    input [15:0] do_out,
    output reg [11:0] led
);
    reg [15:0] cnt; //延时寄存器
    //延时 5ms 改变 XADC 的输出值，以免 XADC 输出不稳定闪动
    always@(posedge clk,posedge rst_n)
        if (!rst_n)
            cnt<=16'd0;
        else if (cnt == 16'd50_000)
            cnt <= 16'd0;
        else if (drdy_out==1)
            cnt<=cnt+1;
    //数据前 12 位输出
    always@(posedge clk,posedge rst_n)
        if (!rst_n)
            led<=12'd0;
        else if (cnt==16'd50_000)
            led<=do_out[15:4];
endmodule

```

7.1.10 二进制转换为 BCD 码输出在数码管上(bin2BCD.v)

```

`timescale 1ns / 1ps
module bin2BCD(
    input [19:0]binary,      //待转换的 2 进制码
    output [31:0]bcd
);
    wire[3:0]bcd_ge,bcd_shi,bcd_bai,bcd_qian,bcd_wan,bcd_shiwan,bcd_baiwan,bcd
_qianwan; //转换为 BCD 码后的个位，十位，百位，千位，万位，十万位，百万位，千
万位。

    wire [19:0] binary2;
    assign binary2=binary*1000000/4095; //转换为 0~1000000 之间的数
    reg[3:0]bcd_ge_o; //灭 0 后个位寄存器
    reg[3:0]bcd_shi_o; //灭 0 后十位寄存器
    reg[3:0]bcd_bai_o; //灭 0 后百位寄存器
    reg[3:0]bcd_qian_o; //灭 0 后千位寄存器
    reg[3:0]bcd_wan_o; //灭 0 后万位寄存器
    reg[3:0]bcd_shiwan_o; //灭 0 后十万位寄存器
    reg[3:0]bcd_baiwan_o; //灭 0 后百万位寄存器
    reg[3:0]bcd_qianwan_o; //灭 0 后千万位寄存器
    //计算得到每一位的数值
    assign    bcd_qianwan=binary2/10000000;
    assign    bcd_baiwan=(binary2%10000000)/1000000;
    assign    bcd_shiwan=(binary2%1000000)/100000;
    assign    bcd_wan=(binary2%100000)/10000;
    assign    bcd_qian=(binary2%10000)/1000;
    assign    bcd_bai=(binary2%1000)/100;
    assign    bcd_shi=(binary2%100)/10;
    assign    bcd_ge=binary2%10;
    //对于每一位除了个位以外，如果高位为 0，进行灭 0 操作
    always@(*)
    begin
        if(bcd_qianwan==0&bcd_baiwan==0&bcd_shiwan==0&bcd_wan==0&bcd_qian==0&
bcd_bai==0&bcd_shi==0)begin
            bcd_qianwan_o=4'he;
            bcd_baiwan_o=4'he;
            bcd_shiwan_o=4'he;
            bcd_wan_o=4'he;
            bcd_qian_o=4'he;
            bcd_bai_o=4'he;

```

```

        bcd_shi_o=4'he;
        bcd_ge_o=bcd_ge;end
    else
if(bcd_qianwan==0&bcd_baiwan==0&bcd_shiwan==0&bcd_wan==0&bcd_qian==0&bcd_
bai==0)begin
        bcd_qianwan_o=4'he;
        bcd_baiwan_o=4'he;
        bcd_shiwan_o=4'he;
        bcd_wan_o=4'he;
        bcd_qian_o=4'he;
        bcd_bai_o=4'he;
        bcd_shi_o=bcd_shi;
        bcd_ge_o=bcd_ge;end
    else
if(bcd_qianwan==0&bcd_baiwan==0&bcd_shiwan==0&bcd_wan==0&bcd_qian==0)begin
        bcd_qianwan_o=4'he;
        bcd_baiwan_o=4'he;
        bcd_shiwan_o=4'he;
        bcd_wan_o=4'he;
        bcd_qian_o=4'he;
        bcd_bai_o=bcd_bai;
        bcd_shi_o=bcd_shi;
        bcd_ge_o=bcd_ge;end
    else
if(bcd_qianwan==0&bcd_baiwan==0&bcd_shiwan==0&bcd_wan==0)begin
        bcd_qianwan_o=4'he;
        bcd_baiwan_o=4'he;
        bcd_shiwan_o=4'he;
        bcd_wan_o=4'he;
        bcd_qian_o=bcd_qian;
        bcd_bai_o=bcd_bai;
        bcd_shi_o=bcd_shi;
        bcd_ge_o=bcd_ge;end
    else if(bcd_qianwan==0&bcd_baiwan==0&bcd_shiwan==0)begin
        bcd_qianwan_o=4'he;
        bcd_baiwan_o=4'he;
        bcd_shiwan_o=4'he;
        bcd_wan_o=bcd_wan;
        bcd_qian_o=bcd_qian;
        bcd_bai_o=bcd_bai;
        bcd_shi_o=bcd_shi;

```

```

        bcd_ge_o=bcd_ge;end
    else if(bcd_qianwan==0&bcd_baiwan==0)begin
        bcd_qianwan_o=4'he;
        bcd_baiwan_o=4'he;
        bcd_shiwan_o=bcd_shiwan;
        bcd_wan_o=bcd_wan;
        bcd_qian_o=bcd_qian;
        bcd_bai_o=bcd_bai;
        bcd_shi_o=bcd_shi;
        bcd_ge_o=bcd_ge;end
    else if(bcd_qianwan==0)begin
        bcd_qianwan_o=4'he;
        bcd_baiwan_o=bcd_baiwan;
        bcd_shiwan_o=bcd_shiwan;
        bcd_wan_o=bcd_wan;
        bcd_qian_o=bcd_qian;
        bcd_bai_o=bcd_bai;
        bcd_shi_o=bcd_shi;
        bcd_ge_o=bcd_ge;end
    else begin
        bcd_qianwan_o=bcd_qianwan;
        bcd_baiwan_o=bcd_baiwan;
        bcd_shiwan_o=bcd_shiwan;
        bcd_wan_o=bcd_wan;
        bcd_qian_o=bcd_qian;
        bcd_bai_o=bcd_bai;
        bcd_shi_o=bcd_shi;
        bcd_ge_o=bcd_ge;end
    end
    //输出灭 0 后的 BCD 码
    assign
bcd={bcd_qianwan_o,bcd_baiwan_o,bcd_shiwan_o,bcd_wan_o,bcd_qian_o,bcd_bai_o,bcd_
shi_o,bcd_ge_o};
endmodule

```

7.1.11 数码管显示模块(show_high_low.v)

```

`timescale 1ns / 1ps
module show_high_low(
    input clk,
    input rst_n,
    input [31:0] bcd, //原始的 BCD 码

```



```

    input frequency_valid,
    input amplitude_valid,
    input offset_valid,
    input duty_valid,
    output [3:0] an1,
    output [3:0] an2,
    output [7:0] a_to_g1,
    output [7:0] a_to_g2
);
    wire [31:0] data_show; //待显示的 BCD 码
//将原始的 BCD 码转换为有波形，参数，单位信息的码(convert.v)
    convert uut_convert(
        .clk(clk),
        .rst_n(rst_n),
        .frequency_valid(frequency_valid),
        .amplitude_valid(amplitude_valid),
        .offset_valid(offset_valid),
        .duty_valid(duty_valid),
        .bcd(bcd),
        .data_show(data_show)
    );
//高四位通过一个 4 位数码管模块显示
    show_4_digit show_high(
        .clk(clk),
        .rst_n(rst_n),
        .x(data_show[31:16]),
        .a_to_g(a_to_g1),
        .an(an1)
    );
//低四位通过一个 4 位数码管模块显示
    show_4_digit show_low(
        .clk(clk),
        .rst_n(rst_n),
        .x(data_show[15:0]),
        .a_to_g(a_to_g2),
        .an(an2)
    );
Endmodule

```

7.1.12 四位数码管显示模块(show_4_digit.v)

```

`timescale 1ns / 1ps

```

```
module show_4_digit(
    input clk,
    input rst_n,
    input [15:0] x, //待显示的 BCD 码
    output reg [7:0] a_to_g, //管段信号
    output reg [3:0] an //位选信号
);
//利用计数器自动溢出的时间控制产生周期约为 5ms 的扫描信号
reg [20:0] clkdiv;
wire [1:0] bitcnt;
reg [3:0] digit;
always@(posedge clk,negedge rst_n)
    if (!rst_n)
        clkdiv <= 21'd0;
    else
        clkdiv <= clkdiv+1;
assign bitcnt = clkdiv[20:19];
//产生位选信号
always@(*)
    if (!rst_n)
        an = 4'd0;
    else
        case(bitcnt)
            2'd0:an = 4'b0001;
            2'd1:an = 4'b0010;
            2'd2:an = 4'b0100;
            2'd3:an = 4'b1000;
            default:an = 4'd0;
        endcase
//分时接收待显示数据
always@(*)
    if (!rst_n)
        digit = 4'd0;
    else
        case(bitcnt)
            2'd0:digit = x[3:0];
            2'd1:digit = x[7:4];
            2'd2:digit = x[11:8];
            2'd3:digit = x[15:12];
            default:digit = 4'd0;
```

```

        endcase
//将待显示的数据 digit 译码为对应的段码信号(共阴极数码管)
always@(*)
    if (!rst_n)
        a_to_g = 8'b11111111;
    else begin
        case(digit)
            0:a_to_g      = 8'b11111100;
            1:a_to_g      = 8'b01100000;
            2:a_to_g      = 8'b11011010;
            3:a_to_g      = 8'b11110010;
            4:a_to_g      = 8'b01100110;
            5:a_to_g      = 8'b10110110;
            6:a_to_g      = 8'b10111110;
            7:a_to_g      = 8'b11100000;
            8:a_to_g      = 8'b11111110;
            9:a_to_g      = 8'b11110110;
            'hA:a_to_g    = 8'b01101110;
            'hB:a_to_g    = 8'b11001110;
            'hC:a_to_g    = 8'b01111100;
            'hD:a_to_g    = 8'b10011110;
            'hE:a_to_g    = 8'b00000000;
            'hF:a_to_g    = 8'b11101110;
            default:a_to_g = 8'b11111100;
        endcase
    end
endmodule

```

7.1.13 将 BCD 码转为有波形，参数，单位等信息的码(convert.v)

```

`timescale 1ns / 1ps
module convert(
    input clk,
    input rst_n,
    input frequency_valid,
    input amplitude_valid,
    input offset_valid,
    input duty_valid,
    input [31:0] bcd,
    output reg [31:0] data_show
);
always@(posedge clk,negedge rst_n)

```

```

        if (!rst_n)
            data_show <= {20'hheeeee,bcd[27:16]};
// 添加频率信息及单位 Hz,若 BCD 码全为 0, 显示 0
        else if (frequency_valid && bcd[27:16]==12'd0)
            data_show <= {4'he,4'he,4'h1,4'h0,4'h0,4'he,4'ha,4'h2};
// 添加幅值信息及单位 U,若 BCD 码全为 0, 显示 0
        else if (amplitude_valid && bcd[27:16]==12'd0)
            data_show <= {4'hb,4'hb,4'he,4'he,4'he,4'h0,4'he,4'hc};
// 添加直流偏移信息及单位 U,若 BCD 码全为 0, 显示
        else if (offset_valid && bcd[27:16]==12'd0)
            data_show <= {4'hb,4'hb,4'he,4'he,4'he,4'h0,4'he,4'hc};
// 添加占空比信息及单位 PER,若 BCD 码全为 0, 显示
        else if (duty_valid && bcd[27:16]==12'd0)
            data_show <= {4'he,4'he,4'he,4'h0,4'he,4'hb,4'hd,4'hf};
// 添加频率信息及单位 Hz
        else if (frequency_valid)
            data_show <= {bcd[27:16],4'h0,4'h0,4'he,4'ha,4'h2};
// 添加幅值信息及单位 U
        else if (amplitude_valid)
            data_show <= {4'hb,4'hb,4'he,bcd[27:16],4'he,4'hc};
// 添加直流偏移信息及单位 U
        else if (offset_valid)
            data_show <= {4'hb,4'hb,4'he,bcd[27:16],4'he,4'hc};
// 添加占空比信息及单位 PER
        else if (duty_valid)
            data_show <= {4'he,bcd[27:16],4'he,4'hb,4'hd,4'hf};
endmodule

```

7.1.14 采样脉冲模块(pulse.v)

```
`timescale 1ns / 1ps
module pulse(
    input clk,
    input rst_n,
    input [11:0] data, //XADC 转换后的数字信号
    input frequency_valid,
    output reg [13:0] frequency,
    output reg sam_pulse
);
parameter total = 100_000_000; //时钟周期 100MHz
parameter sam_freq = 256; //采样频率一个周期 256 个点
```

reg [11:0] period; //周期寄存器
 //将 XADC 转换后的 BCD 码转换为 10 进制数字，并计算发出采样脉冲频率，以及采样后的保持时间

```
always@(posedge clk,negedge rst_n)
  if (!rst_n) begin
    frequency <= 14'd1000;
    period = total/(frequency*sam_freq);end
  else if (data== 12'heee && frequency_valid) begin
    frequency <= 14'd100;
    period = total/(frequency*sam_freq);end
  else if (data[11:8]==4'he && data[7:4]==4'he && frequency_valid) begin
    frequency <= data[3:0]*100;
    period = total/(frequency*sam_freq);end
  else if (data[11:8]==4'he && frequency_valid) begin
    frequency <= data[7:4]*1000+data[3:0]*100;
    period = total/(frequency*sam_freq);end
  else if (frequency_valid) begin
    frequency <= data[11:8]*10_000+ data[7:4]*1000+data[3:0]*100;
    period = total/(frequency*sam_freq);end
```

reg [23:0]cnt; //时间寄存器

//实现从 0 到采样周期的计数

```
always@(posedge clk,negedge rst_n)
begin
  if (!rst_n)
    cnt <= 23'd0;
  else if (cnt == period)
    cnt <= 23'd0;
  else
    cnt <= cnt+1;
end
```

//在每一个采样周期结束时刻发出一个采样脉冲控制信号

```
always@(posedge clk,negedge rst_n)
begin
  if (!rst_n)
    sam_pulse <= 1'd0;
  else if (cnt == period)
    sam_pulse <= 1'd1;
  else
    sam_pulse <= 1'd0;
end
```

```
endmodule
```

7.1.15 数模转换 DAC 波形模块(wave.v)

```
`timescale 1ns / 1ps
```

```
module wave(
```

```
    input clk,
```

```
    input rst_n,
```

```
    input square,
```

```
    input sine,
```

```
    input triangle,
```

```
    input sam_pulse,
```

```
    input offset_valid,
```

```
    input amplitude_valid,
```

```
    input duty_valid,
```

```
    input [11:0]data_XADC,
```

```
    output [7:0] data_out //输出数模转换的数值
```

```
);
```

```
    wire [23:0] data_in; //同时产生方波、正弦波以及三角波，并将三个波形信号汇
```

总在一起

//方波产生模块(square_wave.v)

```
    square_wave uut_square_wave(
```

```
        .clk(clk),
```

```
        .rst_n(rst_n),
```

```
        .square(square),
```

```
        .sam_pulse(sam_pulse),
```

```
        .data(data_in[23:16]), //产生的方波信号，共 8 位
```

```
        .data_XADC(data_XADC),
```

```
        .duty_valid(duty_valid),
```

```
        .amplitude_valid(amplitude_valid),
```

```
        .offset_valid(offset_valid)
```

```
    );
```

//正弦波产生模块(sine_wave.v)

```
    sine_wave uut_sine_wave(
```

```
        .clk(clk),
```

```
        .rst_n(rst_n),
```

```
        .sam_pulse(sam_pulse),
```

```
        .sine(sine),
```

```
        .data_XADC(data_XADC),
```

```
        .data(data_in[15:8]), //产生的正弦波信号，共 8 位
```

```
        .amplitude_valid(amplitude_valid),
```

```

        .offset_valid(offset_valid)
    );
//三角波产生模块(triangle_wave.v)
    triangle_wave uut_triangle_wave(
        .clk(clk),
        .rst_n(rst_n),
        .sam_pulse(sam_pulse),
        .triangle(triangle),
        .data_XADC(data_XADC),
        .data(data_in[7:0]), //产生的三角波信号，共 8 位
        .amplitude_valid(amplitude_valid),
        .offset_valid(offset_valid)
    );
//波形选择，根据波形控制信号，将需要输出的信号输出
    select_wave uut_select_wave(
        .square(square),
        .sine(sine),
        .triangle(triangle),
        .data_in(data_in),
        .data_out(data_out)
    );
endmodule

```

7.1.16 方波产生模块(square_wave.v)

```

`timescale 1ns / 1ps
module square_wave(
    input clk,
    input rst_n,
    input square,
    input sam_pulse,
    input duty_valid,
    input [11:0] data_XADC,
    input amplitude_valid,
    input offset_valid,
    output reg [7:0] data
);
    reg flag; //方波高低电平控制标志
    reg [7:0] cnt; //采样计数器在 0~255 之间计数，对应采样 255 个点
    reg [13:0] duty; //占空比寄存器
//将 XADC 模块，输出的 BCD 码转为 10 进制占空比
    always@(posedge clk,negedge rst_n)

```

```

    if (!rst_n)
        duty <= 14'd50;
    else if (data_XADC[11:0]==12'heee && duty_valid)
        duty <= 14'd0;
    else if (data_XADC[11:8]==4'he && data_XADC[7:4]==4'he && duty_valid)
        duty <= data_XADC[3:0];
    else if (data_XADC[11:8]==4'he && duty_valid)
        duty <= data_XADC[7:4]*10+data_XADC[3:0];
    else if (duty_valid)
        duty<=data_XADC[11:8]*100+data_XADC[7:4]*10+data_XADC[3:0];
    reg [13:0] amplitude; //幅值寄存器
    //将 XADC 模块，输出的 BCD 码转为 10 进制幅值
    always@(posedge clk,negedge rst_n)
        if (!rst_n)
            amplitude <= 14'd50;
        else if (data_XADC[11:0]==12'heee && amplitude_valid)
            amplitude <= 14'd0;
        else if (data_XADC[11:8]==4'he && data_XADC[7:4]==4'he &&
amplitude_valid)
            amplitude <= data_XADC[3:0];
        else if (data_XADC[11:8]==4'he && amplitude_valid)
            amplitude <= data_XADC[7:4]*10+data_XADC[3:0];
        else if (amplitude_valid)
            amplitude<=data_XADC[11:8]*100+data_XADC[7:4]*10+data_XADC[3:0];

    reg [13:0] offset; //偏移值寄存器
    //将 XADC 模块，输出的 BCD 码转为 10 进制偏移值
    always@(posedge clk,negedge rst_n)
        if (!rst_n)
            offset <= 14'd0;
        else if (data_XADC[11:0]==12'heee && offset_valid)
            offset <= 14'd0;
        else if (data_XADC[11:8]==4'he&&data_XADC[7:4]==4'he && offset_valid)
            offset <= data_XADC[3:0]*255*(100-amplitude)/10000;
        else if (data_XADC[11:8]==4'he && offset_valid)
            offset<=(data_XADC[7:4]*10+data_XADC[3:0])*255*(100-amplitude)/10000;
        else if (offset_valid)
            offset<=(data_XADC[11:8]*100+data_XADC[7:4]*10+data_XADC[3:0])*255*(100-a
mplitude)/10000;
    //采样寄存器在 0~255 之间循环计数

```



```

always@ (posedge clk , negedge rst_n)
    if (!rst_n)
        cnt <= 8'd1;
    else if (square && sam_pulse && cnt==255)
        cnt <= 8'd1;
    else if (square && sam_pulse)
        cnt <= cnt + 1;
//在一个周期*占空比之前设置标志为 0，之后标志 flag 为 1
always@ (posedge clk , negedge rst_n)
    if (!rst_n)
        flag <= 0;
    else if (square && sam_pulse && cnt<=255*duty/100)
        flag <= 1'b0;
    else if (square && sam_pulse)
        flag <= 1'b1;
//根据标志的不同对于方波占空比实现翻转，并考虑了幅值的改变与直流偏移
always@ (posedge clk , negedge rst_n)
    if (!rst_n)
        data <= 8'd0;
    else if (square && sam_pulse && flag)
        data <= 8'd0+offset;
    else if (square && sam_pulse && ~flag)
        data <= 8'd255*amplitude/100+offset;
endmodule

```

7.1.17 正弦波产生模块(sine_wave.v)

```

`timescale 1ns / 1ps
module sine_wave(
    input clk,
    input rst_n,
    input sine,
    input sam_pulse,
    input [11:0] data_XADC,
    input amplitude_valid,
    input offset_valid,
    output wire [7:0] data
);
    wire [7:0] data_in; //rom IP 核中直接输出的信号
    reg [7:0] addr; //rom 地址
    reg [13:0] amplitude; //幅值寄存器
//将 XADC 模块，输出的 BCD 码转为 10 进制幅值

```

```

always@(posedge clk,negedge rst_n)
    if (!rst_n)
        amplitude <= 14'd50;
    else if (data_XADC[11:0]==12'heee && amplitude_valid)
        amplitude <= 14'd0;
    else if (data_XADC[11:8]==4'he && data_XADC[7:4]==4'he &&
amplitude_valid)
        amplitude <= data_XADC[3:0];
    else if (data_XADC[11:8]==4'he && amplitude_valid)
        amplitude <= data_XADC[7:4]*10+data_XADC[3:0];
    else if (amplitude_valid)
amplitude<=data_XADC[11:8]*100+data_XADC[7:4]*10+data_XADC[3:0];

reg [13:0] offset; //偏移值寄存器
//将 XADC 模块，输出的 BCD 码转为 10 进制偏移值
always@(posedge clk,negedge rst_n)
    if (!rst_n)
        offset <= 14'd0;
    else if (data_XADC[11:0]==12'heee && offset_valid)
        offset <= 14'd0;
    else if(data_XADC[11:8]==4'he&&data_XADC[7:4]==4'he && offset_valid)
        offset <= data_XADC[3:0]*255*(100-amplitude)/10000;
    else if (data_XADC[11:8]==4'he && offset_valid)
offset<=(data_XADC[7:4]*10+data_XADC[3:0])*255*(100-amplitude)/10000;
    else if (offset_valid)
offset<=(data_XADC[11:8]*100+data_XADC[7:4]*10+data_XADC[3:0])*255*(100-a
mplitude)/10000;

always@ (posedge clk , negedge rst_n)
    if (!rst_n)
        addr <= 0; //复位地址清零
    else if(sine && sam_pulse)
        addr <= addr + 1; //地址自增
//调用 ROM 的 IP 核
dist_mem_gen_0 u_sin_wave (
    .a(addr),          // input wire [7 : 0] a
    .clk(clk),         // input wire clk
    .qspo(data_in)     // output wire [7 : 0] qspo
);
assign data = data_in*amplitude/100 + offset; //直接输出的信号加上直流偏移
endmodule

```

7.1.18 三角波产生模块(triangle.v)

```

`timescale 1ns / 1ps
module triangle_wave(
    input clk,
    input rst_n,
    input triangle,
    input sam_pulse,
    input [11:0] data_XADC,
    input amplitude_valid,
    input offset_valid,
    output wire [7:0] data
);
    reg flag; //三角波上升下降控制标志
    reg [7:0] cnt; //采样计数器在 0~255 之间计数，对应采样 255 个点
    reg [7:0] data_in;
    reg [13:0] amplitude; //幅值寄存器
    //将 XADC 模块，输出的 BCD 码转为 10 进制幅值
    always@(posedge clk,negedge rst_n)
        if (!rst_n)
            amplitude <= 14'd50;
        else if (data_XADC[11:0]==12'heee && amplitude_valid)
            amplitude <= 14'd0;
        else if (data_XADC[11:8]==4'he && data_XADC[7:4]==4'he &&
amplitude_valid)
            amplitude <= data_XADC[3:0];
        else if (data_XADC[11:8]==4'he && amplitude_valid)
            amplitude <= data_XADC[7:4]*10+data_XADC[3:0];
        else if (amplitude_valid)
            amplitude<=data_XADC[11:8]*100+data_XADC[7:4]*10+data_XADC[3:0];

    reg [13:0] offset; //偏移值寄存器
    //将 XADC 模块，输出的 BCD 码转为 10 进制偏移值
    always@(posedge clk,negedge rst_n)
        if (!rst_n)
            offset <= 14'd0;
        else if (data_XADC[11:0]==12'heee && offset_valid)
            offset <= 14'd0;
        else if(data_XADC[11:8]==4'he&&data_XADC[7:4]==4'he && offset_valid)
            offset <= data_XADC[3:0]*255*(100-amplitude)/10000;
        else if (data_XADC[11:8]==4'he && offset_valid)

```

```

offset<=(data_XADC[7:4]*10+data_XADC[3:0])*255*(100-amplitude)/10000;
    else if (offset_valid)
offset<=(data_XADC[11:8]*100+data_XADC[7:4]*10+data_XADC[3:0])*255*(100-a
mplitude)/10000;
    assign data = data_in * amplitude/100 + offset; //输出数据改变幅值，加直流偏移
//采样寄存器在 0~255 之间循环计数
    always@ (posedge clk , negedge rst_n)
        if (!rst_n)
            cnt <= 8'd0;
        else if (triangle && sam_pulse && cnt==255)
            cnt <= 8'd0;
        else if (triangle && sam_pulse)
            cnt <= cnt + 1;
//在一个周期*占空比之前设置标志为 0，之后标志 flag 为 1
    always@ (posedge clk , negedge rst_n)
        if (!rst_n)
            flag <= 0;
        else if (triangle && sam_pulse && cnt<127)
            flag <= 1'b0;
        else if (triangle && sam_pulse && cnt == 8'hff)
            flag <= 1'b0;
        else if (triangle && sam_pulse)
            flag <= 1'b1;
//在前半个周期输出数据不断上升，后半周期输出数据不断下降
    always@ (posedge clk , negedge rst_n)
        if (!rst_n)
            data_in <= 8'd0;
        else if (triangle && sam_pulse && flag)
            data_in <= 8'd255 - 2*cnt;
        else if (triangle && sam_pulse && ~flag)
            data_in <= 2*cnt;
endmodule

```

7.1.19 波形选择模块(select.v)

```

`timescale 1ns / 1ps
module select_wave(
    input square,
    input sine,
    input triangle,
    input [23:0] data_in,
    output reg [7:0] data_out

```

```

);
//根据波形控制信号选择输出相应的波形数据
always@(*)
    if (square==1 && sine==0 && triangle==0)
        data_out = data_in[23:16];
    else if (square==0 && sine==1 && triangle==0)
        data_out = data_in[15:8];
    else if (square==0 && sine==0 && triangle==1)
        data_out = data_in[7:0];
    else
        data_out = data_in[23:16];
endmodule

```

7.2 约束文件(XADC_voltage.xdc)

```

set_property -dict {PACKAGE_PIN B12 IOSTANDARD LVCMOS33} [get_ports vauxn1]
set_property -dict {PACKAGE_PIN C12 IOSTANDARD LVCMOS33} [get_ports vauxp1]
set_property -dict {PACKAGE_PIN P17 IOSTANDARD LVCMOS33} [get_ports clk]
set_property -dict {PACKAGE_PIN P15 IOSTANDARD LVCMOS33} [get_ports rst_n]

set_property -dict {PACKAGE_PIN G4 IOSTANDARD LVCMOS33} [get_ports {square}]
set_property -dict {PACKAGE_PIN G3 IOSTANDARD LVCMOS33} [get_ports {sine}]
set_property -dict {PACKAGE_PIN J4 IOSTANDARD LVCMOS33} [get_ports {triangle}]
set_property -dict {PACKAGE_PIN H4 IOSTANDARD LVCMOS33} [get_ports
{frequency_valid}]
set_property -dict {PACKAGE_PIN J3 IOSTANDARD LVCMOS33} [get_ports {amplitude_valid}]
set_property -dict {PACKAGE_PIN J2 IOSTANDARD LVCMOS33} [get_ports {offset_valid}]
set_property -dict {PACKAGE_PIN K2 IOSTANDARD LVCMOS33} [get_ports {duty_valid}]

set_property -dict {PACKAGE_PIN G1 IOSTANDARD LVCMOS33} [get_ports {an2[3]}]
set_property -dict {PACKAGE_PIN F1 IOSTANDARD LVCMOS33} [get_ports {an2[2]}]
set_property -dict {PACKAGE_PIN E1 IOSTANDARD LVCMOS33} [get_ports {an2[1]}]
set_property -dict {PACKAGE_PIN G6 IOSTANDARD LVCMOS33} [get_ports {an2[0]}]

set_property -dict {PACKAGE_PIN G2 IOSTANDARD LVCMOS33} [get_ports {an1[3]}]
set_property -dict {PACKAGE_PIN C2 IOSTANDARD LVCMOS33} [get_ports {an1[2]}]
set_property -dict {PACKAGE_PIN C1 IOSTANDARD LVCMOS33} [get_ports {an1[1]}]
set_property -dict {PACKAGE_PIN H1 IOSTANDARD LVCMOS33} [get_ports {an1[0]}]

set_property -dict {PACKAGE_PIN D4 IOSTANDARD LVCMOS33} [get_ports {a_to_g2[7]}]
set_property -dict {PACKAGE_PIN E3 IOSTANDARD LVCMOS33} [get_ports {a_to_g2[6]}]
set_property -dict {PACKAGE_PIN D3 IOSTANDARD LVCMOS33} [get_ports {a_to_g2[5]}]
set_property -dict {PACKAGE_PIN F4 IOSTANDARD LVCMOS33} [get_ports {a_to_g2[4]}]
set_property -dict {PACKAGE_PIN F3 IOSTANDARD LVCMOS33} [get_ports {a_to_g2[3]}]

```

```
set_property -dict {PACKAGE_PIN E2 IOSTANDARD LVCMOS33} [get_ports {a_to_g2[2]}]
set_property -dict {PACKAGE_PIN D2 IOSTANDARD LVCMOS33} [get_ports {a_to_g2[1]}]
set_property -dict {PACKAGE_PIN H2 IOSTANDARD LVCMOS33} [get_ports {a_to_g2[0]}]
```

```
set_property -dict {PACKAGE_PIN B4 IOSTANDARD LVCMOS33} [get_ports {a_to_g1[7]}]
set_property -dict {PACKAGE_PIN A4 IOSTANDARD LVCMOS33} [get_ports {a_to_g1[6]}]
set_property -dict {PACKAGE_PIN A3 IOSTANDARD LVCMOS33} [get_ports {a_to_g1[5]}]
set_property -dict {PACKAGE_PIN B1 IOSTANDARD LVCMOS33} [get_ports {a_to_g1[4]}]
set_property -dict {PACKAGE_PIN A1 IOSTANDARD LVCMOS33} [get_ports {a_to_g1[3]}]
set_property -dict {PACKAGE_PIN B3 IOSTANDARD LVCMOS33} [get_ports {a_to_g1[2]}]
set_property -dict {PACKAGE_PIN B2 IOSTANDARD LVCMOS33} [get_ports {a_to_g1[1]}]
set_property -dict {PACKAGE_PIN D5 IOSTANDARD LVCMOS33} [get_ports {a_to_g1[0]}]
```

```
set_property -dict {PACKAGE_PIN R5 IOSTANDARD LVCMOS33} [get_ports ILE]
set_property -dict {PACKAGE_PIN N6 IOSTANDARD LVCMOS33} [get_ports CS_n]
set_property -dict {PACKAGE_PIN V6 IOSTANDARD LVCMOS33} [get_ports WR1_n]
set_property -dict {PACKAGE_PIN R6 IOSTANDARD LVCMOS33} [get_ports WR2_n]
set_property -dict {PACKAGE_PIN V7 IOSTANDARD LVCMOS33} [get_ports XFER_n]
```

```
set_property -dict {PACKAGE_PIN T8 IOSTANDARD LVCMOS33} [get_ports {data_wave[0]}]
set_property -dict {PACKAGE_PIN R8 IOSTANDARD LVCMOS33} [get_ports {data_wave[1]}]
set_property -dict {PACKAGE_PIN T6 IOSTANDARD LVCMOS33} [get_ports {data_wave[2]}]
set_property -dict {PACKAGE_PIN R7 IOSTANDARD LVCMOS33} [get_ports {data_wave[3]}]
set_property -dict {PACKAGE_PIN U6 IOSTANDARD LVCMOS33} [get_ports {data_wave[4]}]
set_property -dict {PACKAGE_PIN U7 IOSTANDARD LVCMOS33} [get_ports {data_wave[5]}]
set_property -dict {PACKAGE_PIN V9 IOSTANDARD LVCMOS33} [get_ports {data_wave[6]}]
set_property -dict {PACKAGE_PIN U9 IOSTANDARD LVCMOS33} [get_ports {data_wave[7]}]
```

```
set_property -dict {PACKAGE_PIN R11 IOSTANDARD LVCMOS33} [get_ports {key_offset}]
set_property -dict {PACKAGE_PIN R17 IOSTANDARD LVCMOS33} [get_ports {key_duty}]
set_property -dict {PACKAGE_PIN R15 IOSTANDARD LVCMOS33} [get_ports {key_wave}]
set_property -dict {PACKAGE_PIN V1 IOSTANDARD LVCMOS33} [get_ports
{key_frequency}]
set_property -dict {PACKAGE_PIN U4 IOSTANDARD LVCMOS33} [get_ports
{key_amplitude}]
```

```
set_property PULLUP true [get_ports ILE]
set_property PULLDOWN true [get_ports WR2_n]
set_property PULLDOWN true [get_ports CS_n]
set_property PULLDOWN true [get_ports XFER_n]
set_property PULLDOWN true [get_ports WR1_n]
```